

Module 3

Unicast Routing

Introduction

- In an internet, the goal of the network layer is to deliver a datagram from its source to its destination or destinations.
- If a datagram is destined for only one destination (one-to-one delivery), we have **unicast routing**.
- If the datagram is destined for several destinations (one-to-many delivery), we have **multicast routing**.
- The routing can be possible if a router has a forwarding table (to forward a packet to the appropriate next node)
- To make the **forwarding tables** of the router, the Internet **needs routing protocols** that will be active all the time in the background and update the forwarding tables.

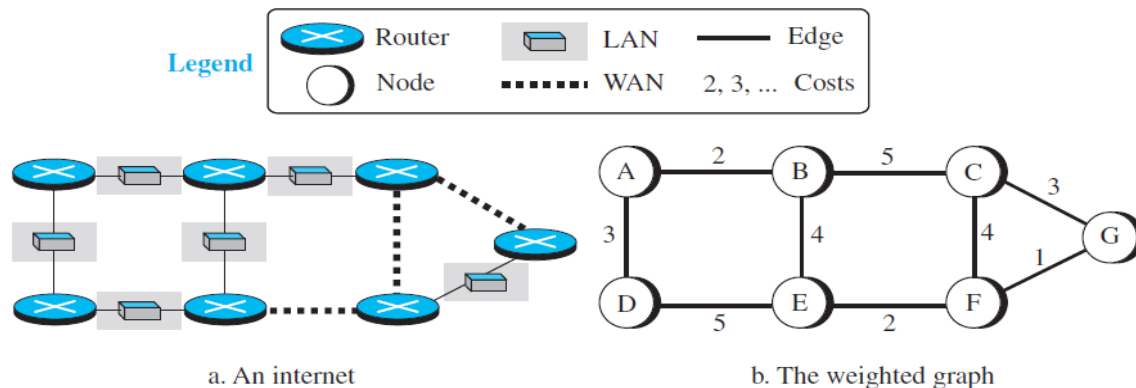
A routing protocol is a combination of rules and procedures that lets routers in the Internet inform each other of changes.

General Idea

- In unicast routing, a packet is routed, hop by hop, from its source to its destination by the help of forwarding tables
- The source host needs no forwarding table because it delivers its packet to the default router in its local network.
- The destination host needs no forwarding table either because it receives the packet from its default router in its local network.
- This means that only the routers that glue together the networks in the internet need forwarding tables
- There are **several routes that a packet can travel from the source to the destination**; what must be determined is which route the packet should take.

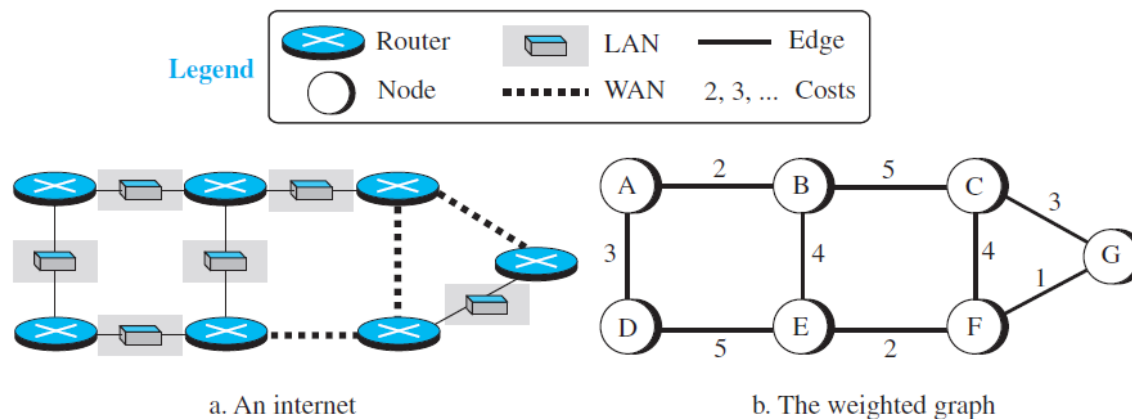
An Internet as a Graph

- To find the best route, an internet can be modeled as a graph.
- Set of nodes (routers) and edges (each network between a pair of routers)
- A *weighted graph*, in which each edge is associated with a cost.
- In routing, however, the cost of an edge has a different interpretation in different routing protocols (bandwidth, delay, time, cost, number of hops etc)
- If there is no edge between the nodes, the cost is infinity.



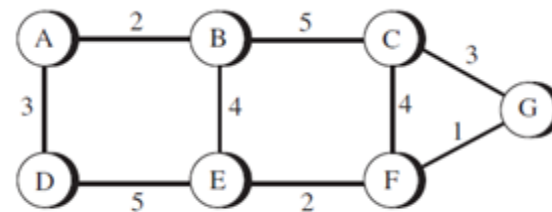
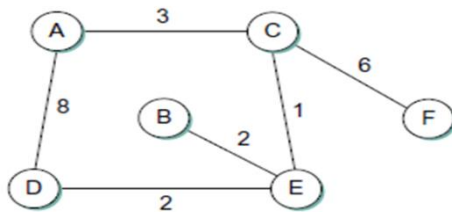
Least-Cost Routing

- *The best* route from the source router to the destination router is to find the *least cost* between the two.
- In Figure, the best route between A and E is A-B-E, with the cost of 6.
- This means that each router needs to find the least-cost route between itself and all the other routers to be able to route a packet using this criteria.



Least-Cost Trees

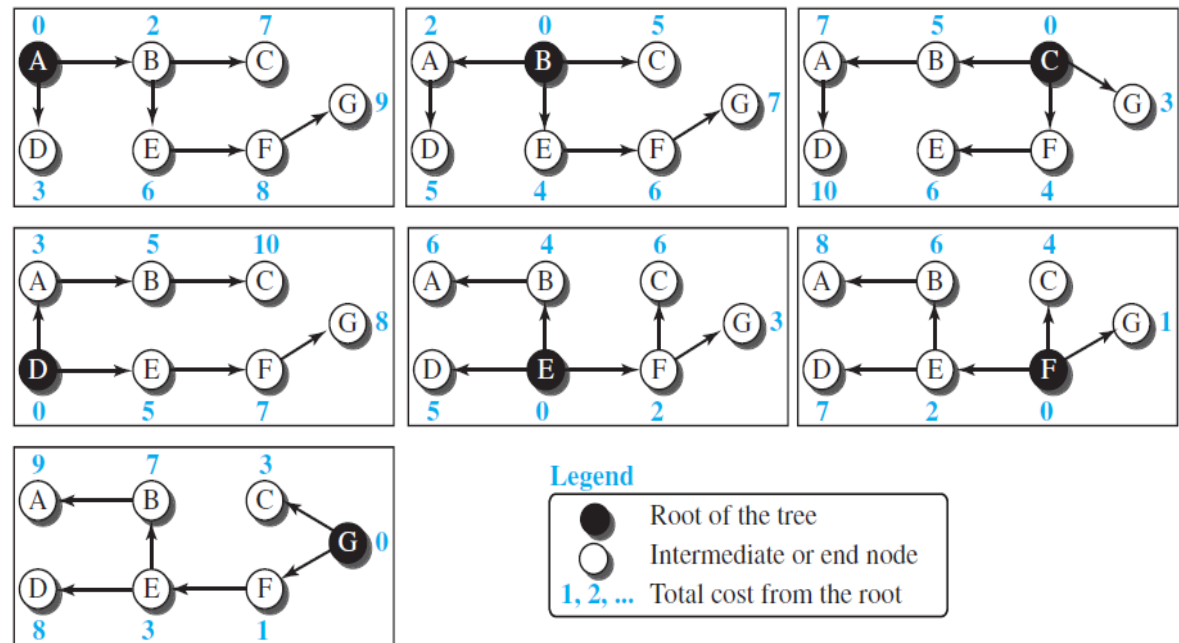
- If there are N routers in an internet, there are $(N - 1)$ least-cost paths from each router to any other router. This means we need $N \times (N - 1)$ least-cost paths for the whole internet
- A least-cost tree is a tree with the source router as the root that spans the whole graph (visits all other nodes) and in which the path between the root and any other node is the **shortest**.
- In this way, we can have only one shortest-path tree for each node; we have N least-cost trees for the whole internet



b. The weighted graph

Least-Cost Trees

Figure 20.2 Least-cost trees for nodes in the internet of Figure 20.1



The least-cost trees for a weighted graph can have several properties if they are created using consistent criteria.

1. The least-cost route from X to Y in X's tree is the inverse of the least-cost route from Y to X in Y's tree; the cost in both directions is the same

The route from A to F in A's tree is (A → B → E → F),

The route from F to A in F's tree is (F → E → B → A)

The cost is 8 in each case

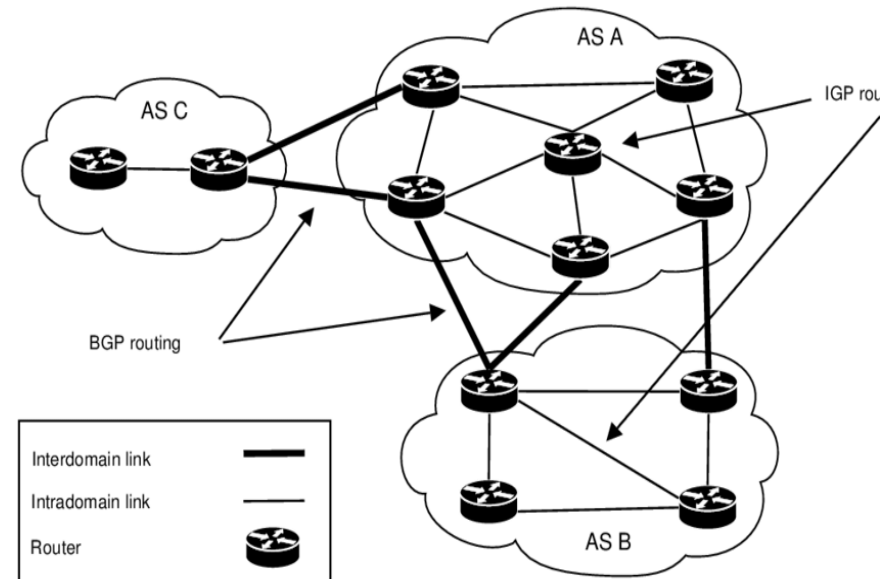
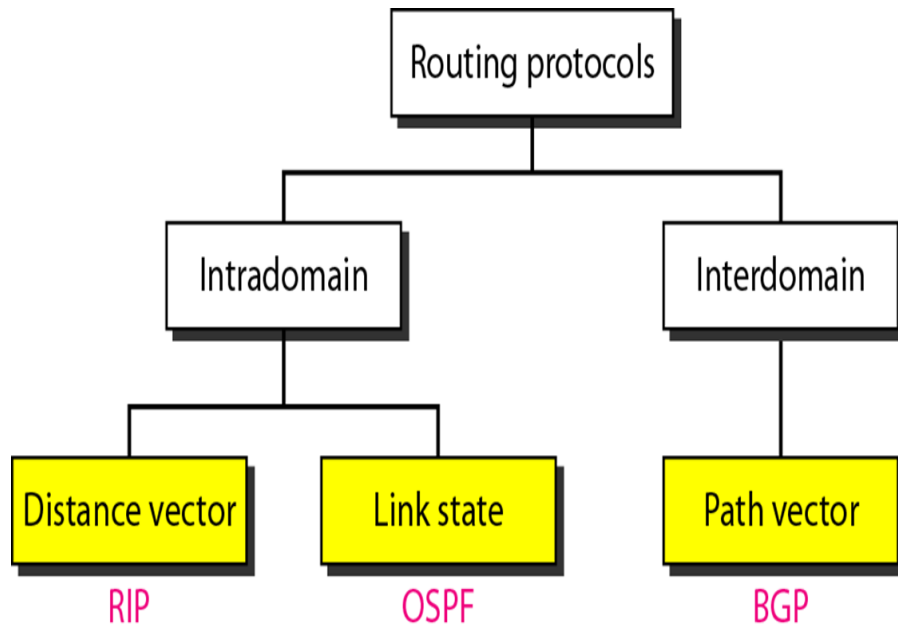
Least-Cost Trees

2. Instead of travelling from X to Z using X's tree, we can travel from X to Y using X's tree and continue from Y to Z using Y's tree.

- From A to G in A's tree using the route $(A \rightarrow B \rightarrow E \rightarrow F \rightarrow G)$
- A to E in A's tree $(A \rightarrow B \rightarrow E)$
- E's tree using the route $(E \rightarrow F \rightarrow G)$.
- The cost in the first case is 9; the cost in the second case is also 9 $(6 + 3)$.

ROUTING ALGORITHMS

- Several routing algorithms have been designed in the past.
- The differences between these methods are in the way they interpret the least cost and the way they create the least-cost tree for each node
- Here, we discuss the common algorithms; later we show how a routing protocol in the Internet implements one of these algorithms.



Distance Vector/ Bellman-ford routing

- Goal: To find the best route (least cost path).

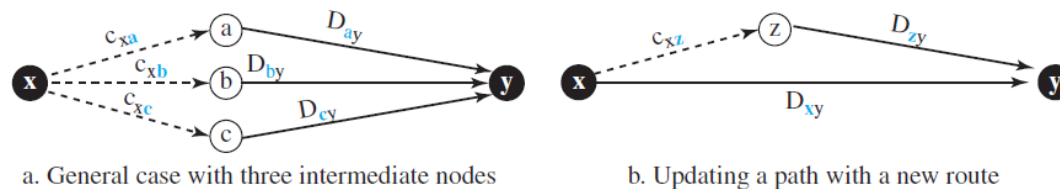
Operation:

- Each Node creates its own least-cost tree with the basic information it has about its immediate neighbors.
- The incomplete trees are exchanged between **immediate neighbors** to make the trees more and more complete and to represent the whole internet.
- In distance-vector routing, a router continuously tells all of its neighbors what it knows about the whole internet
- Distance vector uses:
 - *Bellman-Ford Equation*
 - *Distance Vectors*

Bellman-Ford equation

- The heart of distance-vector routing is the famous **Bellman-Ford** equation
- This equation is used to find the least cost (shortest distance) between a source node, x , and a destination node, y , through some intermediary nodes ($a, b, c, \dots$)
- In distance-vector routing, normally we want to update an existing least cost with a least cost through an intermediary node,

Figure 20.3 Graphical idea behind Bellman-Ford equation

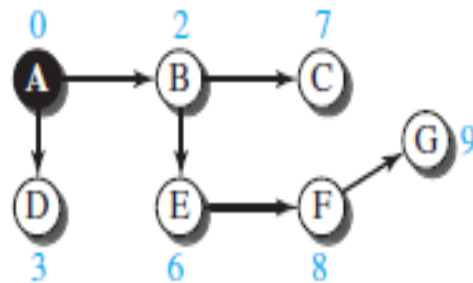


$$D_{xy} = \min \{ (c_{xa} + D_{ay}), (c_{xb} + D_{by}), (c_{xc} + D_{cy}), \dots \}$$

$$D_{xy} = \min \{ D_{xy}, (c_{xz} + D_{zy}) \}$$

Distance Vector

- Distance vector is the rationale for the name *distance-vector routing*.
- Each node constructs a one-dimensional array containing the "distances"(costs) to all other nodes and distributes that vector to its immediate neighbors



a. Tree for node A

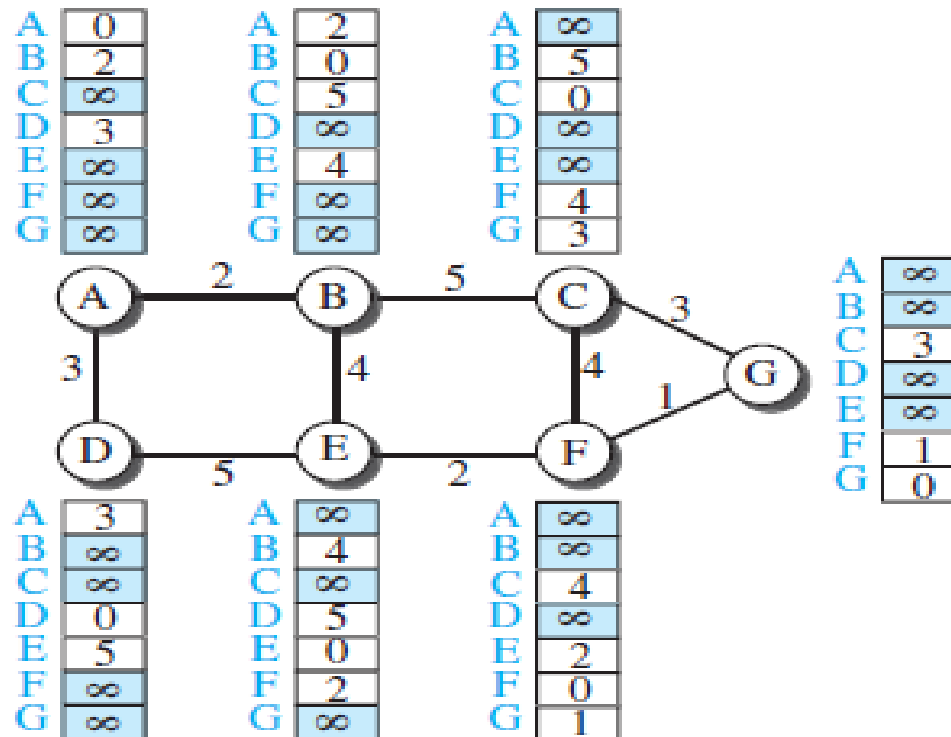
A	
A	0
B	2
C	7
D	3
E	6
F	8
G	9

b. Distance vector for node A

Distance Vector Algorithm Steps

- Each node constructs a one-dimensional array containing the "distances"(costs) to all other.
- After each node has created its vector, it sends a copy of the vector to all its immediate neighbors
- After a node receives a distance vector from a neighbor, it updates its distance vector using the Bellman-Ford equation

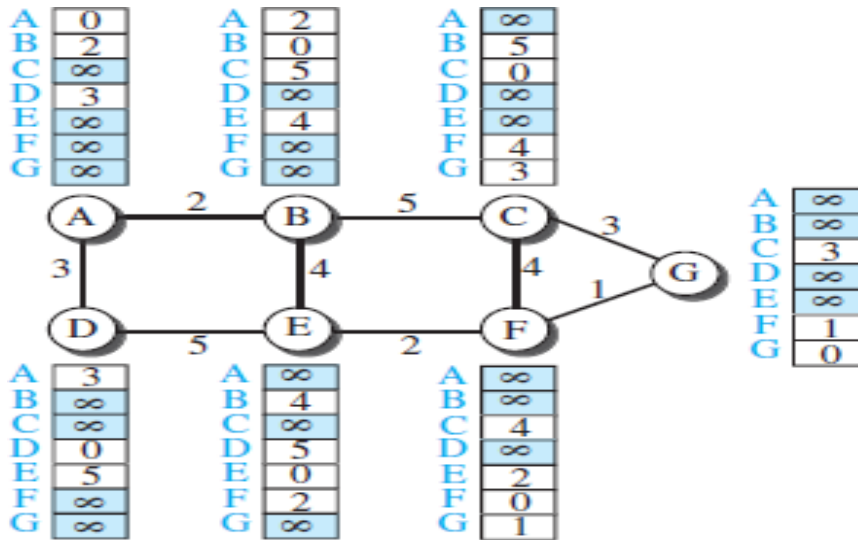
Distance Vector Routing (Bellman-Ford routing algorithm)



Each node knows the cost of the link to each of its directly connected neighbors

Basic vectors cannot help the internet to effectively forward a packet.

Distance Vector Routing (Bellman-Ford routing algorithm)



A	2
B	0
C	5
D	5
E	4
F	6
G	Infi

A	infi
B	5
C	0
D	infi
E	infi
F	4
G	3

$$= \text{Min} (B [], 5 + C [])$$

Figure 20.6 Updating distance vectors

New B	Old B	A
A 2	A 2	A 0
B 0	B 0	B 2
C 5	C 5	C ∞
D 5	D ∞	D 3
E 4	E 4	E ∞
F ∞	F ∞	F ∞
G ∞	G ∞	G ∞

$B [] = \text{min} (B [], 2 + A [])$

a. First event: B receives a copy of A's vector.

New B	Old B	E
A 2	A 2	A ∞
B 0	B 0	B 4
C 5	C 5	C ∞
D 5	D 5	D 5
E 4	E 4	E 0
F 6	F ∞	F 2
G ∞	G ∞	G ∞

$B [] = \text{min} (B [], 4 + E [])$

b. Second event: B receives a copy of E's vector.

Note:
X[]: the whole vector

B Distance vector

A	2
B	0
C	5
D	5
E	4
F	6
G	8

Distance Vector Routing

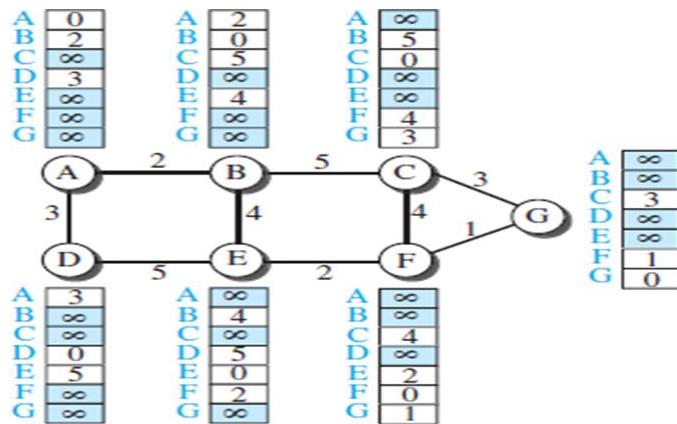
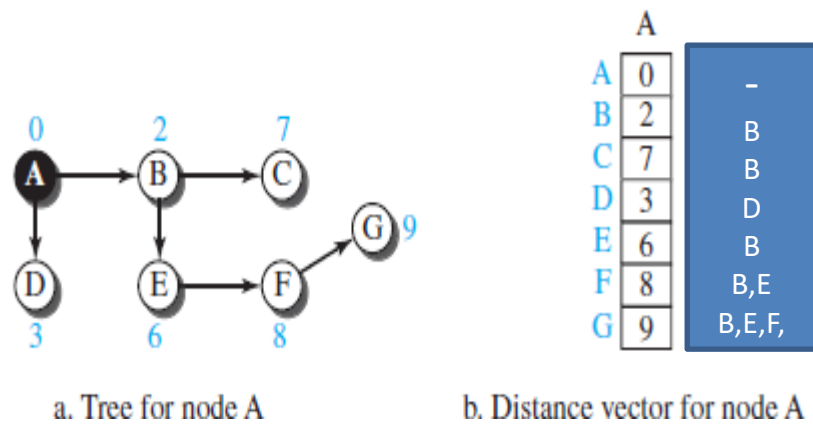


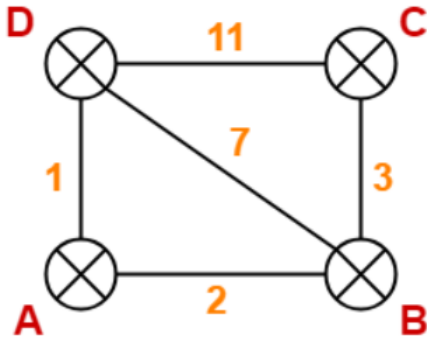
Figure 20.4 The distance vector corresponding to a tree



Distance Vector Routing Algorithm

```
1 Distance_Vector_Routing ( )
2 {
3     // Initialize (create initial vectors for the node)
4     D[myself] = 0
5     for (y = 1 to N)
6     {
7         if (y is a neighbor)
8             D[y] = c[myself][y]
9         else
10            D[y] =  $\infty$ 
11    }
12    send vector {D[1], D[2], ..., D[N]} to all neighbors
13    // Update (improve the vector with the vector received from a neighbor)
14    repeat (forever)
15    {
16        wait (for a vector  $D_w$  from a neighbor w or any change in the link)
17        for (y = 1 to N)
18        {
19            D[y] = min [D[y], (c[myself][w] +  $D_w[y]$ )]    // Bellman-Ford equation
20        }
21        if (any change in the vector)
22            send vector {D[1], D[2], ..., D[N]} to all neighbors
23    }
24 } // End of Distance Vector
```

Consider a network with 5 nodes (A-D). The network uses Distance Vector Routing Protocol. Initially, each node generated its own distance vector and exchanges their distance vector with their respective neighbours. After this exchange, build the routing table at the node A for the same.



A's Neighbours – B and D

A Initial Table

A	0
B	2
C	∞
D	1

B Initial Table

A	2
B	0
C	3
D	7

D Initial Table

A	1
B	7
C	11
D	0

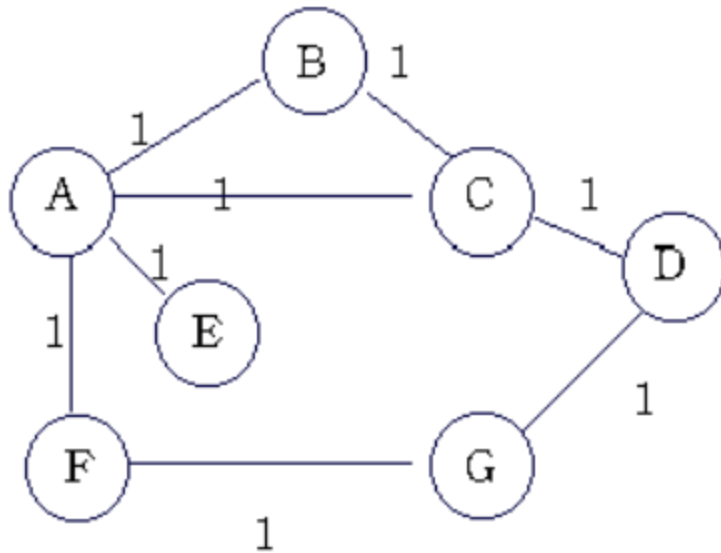
Updated A'S Table

A	0	
B	2	
C	5	B
D	1	

$$A[] = \min(A[], 2+B[])$$

$$A[] = \min(A[], 1+D[])$$

Distance Vector

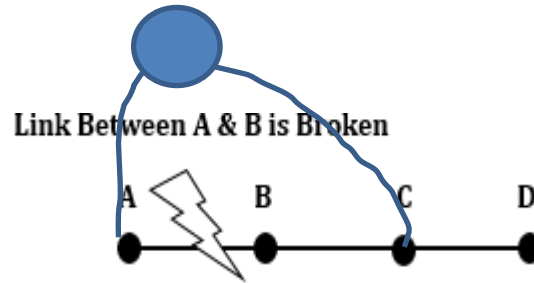


Initial distances stored at each node
final distances stored at each node

Count to Infinity Problem

- It is an issue in Distance Vector Routing
- Counting to infinity is just another name for a **routing loop**
- A problem with distance-vector routing is that any decrease in cost (good news) propagates quickly, but any increase in cost (bad news) will propagate slowly.
- For a routing protocol to work properly, if a link is broken (cost becomes infinity), every other router should be aware of it immediately, but in distance-vector routing, this takes some time. The problem is referred to as *count to infinity*.

Count to Infinity Problem



	A	B	C	D
A	0, -	1, A	2, B	3, C
B	1, B	0, -	2, C	3, D
C	2, B	1, C	0, -	1, D
D	3, B	2, C	1, D	0, -

B Routing Table

(When A to B link is cut)

A	∞
B	0
C	1
D	2

C Routing Table

A	2
B	1
C	0
D	1

B Routing Table
(After receiving from C)

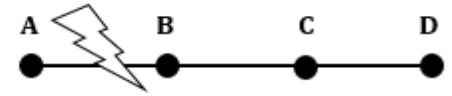
A	3
B	0
C	1
D	2

C Routing Table
(After receiving from B)

A	4
B	1
C	0
D	1

Count to Infinity Problem

Link Between A & B is Broken



	A	B	C	D
A	0, -	1, A	2, B	3, C
B	1, B	0, -	2, C	3, D
C	2, B	1, C	0, -	1, D
D	3, B	2, C	1, D	0, -

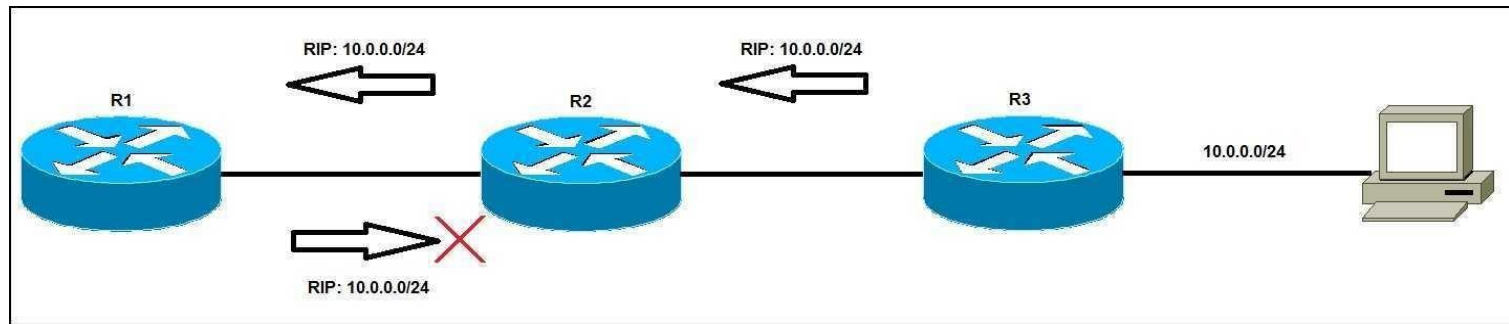
- There is only one link between A and the other parts of the network.
- Now imagine that the link between A and B is cut.
- At this time, B corrects its table.
- After a specific amount of time, routers exchange their tables, and so B receives C's routing table.
- Since C doesn't know what has happened to the link between A and B, it says that it has a link to A with the weight of 2 (1 for C to B, and 1 for B to A -- it doesn't know B has no link to A).
- B receives this table and thinks there is a separate link between C and A, so it corrects its table and changes infinity to 3 (1 for B to C, and 2 for C to A, as C said).
- Once again, routers exchange their tables.
- When C receives B's routing table, it sees that B has changed the weight of its link to A from 1 to 3, so C updates its table and changes the weight of the link to A to 4 (1 for C to B, and 3 for B to A, as B said).
- This process loops until all nodes find out that the weight of link to A is infinity.

Count to Infinity Problem

- One way to solve this problem is for routers to send information only to the neighbors that are not exclusive links to the destination.
- For example, in this case, C shouldn't send any information to B about A, because B is the only way to A

Split horizon

- A method used by distance vector routing protocols to prevent a routing loop in a network.
- The principle is simple – a router will not advertise a route back onto the interface from which it was learned.

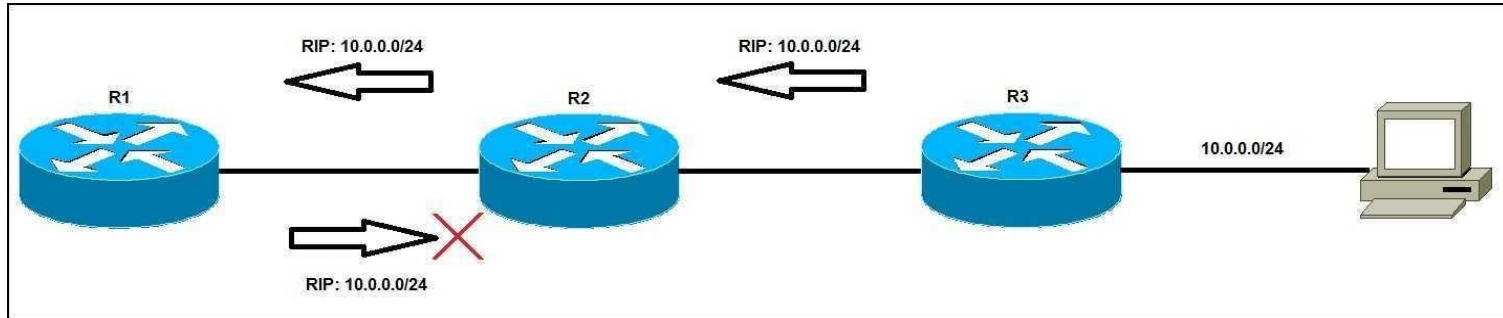


without the split horizon mechanism

We have a network of three routers. All routers are running RIP, a distance vector protocol. R3 is directly connected to the 10.0.0.0/24 network and advertises that network using RIP to R2. R2 receives the routing update, places the route in its routing table and informs R1 about the 10.0.0.0/24.

Because the split horizon mechanism is enabled by default on all interfaces, R1 will not advertise to R2 that it has the route to 10.0.0.0/24

Split horizon

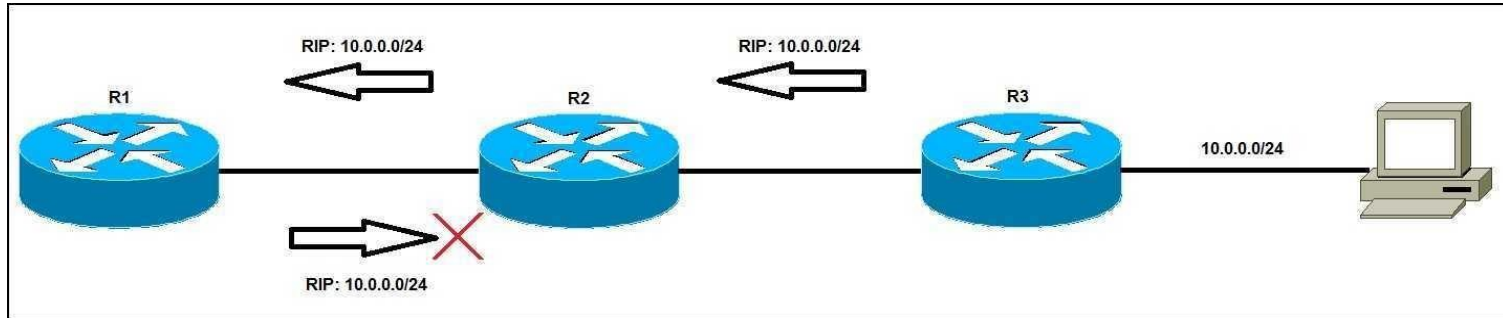


R1 will not advertise to R2 that it has the route to 10.0.0.0/24

Using the split-horizon strategy has one drawback. Normally, the corresponding protocol uses a timer (30 s), and if there is no news about a route, the node deletes the route from its table.

When node R1 in the previous scenario eliminates the route to R3 from its advertisement to R2, node R2 cannot guess whether this is due to the split-horizon strategy (the source of information was R2) or because R1 has not received any news about R3 recently.

Split horizon



R1 will not advertise to R2 that it has the route to 10.0.0.0/24

Using the split-horizon strategy has one drawback. Normally, the corresponding protocol uses a timer (30 s), and if there is no news about a route, the node deletes the route from its table.

When node R1 in the previous scenario eliminates the route to R3 from its advertisement to R2, node R2 cannot guess whether this is due to the split-horizon strategy (the source of information was R2) or because R1 has not received any news about R3 recently.

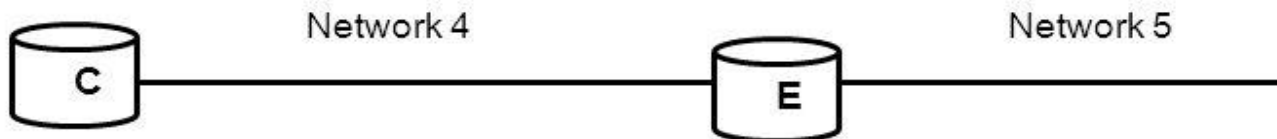
Poison Reverse

- A method to prevent routing loops within computer networks
- Distance-vector routing protocols in computer networks use route poisoning **to indicate to other routers that a route is no longer reachable and should be removed from their routing tables.**
- When the protocol detects an invalid route, all of the routers in the network are informed that the bad route has **an infinite (∞) route metric.**
- In the poison reverse strategy R1 can still advertise the value for R3, but if the source of information is R2, it can replace the distance with infinity as a warning: “Do not use this value; what I know about this route comes from you.”

Poison Reverse

Poison Reverse –

A route that is not 'good' is sent a **poison reverse** which removes the route



When Network 5 goes down, Router E initiates route poisoning by entering a table entry for Network 5 as 16, for RIP, **unreachable**.

By this poisoning of the route to Network 5, Router C is not susceptible to incorrect updates about the route to Network 5.

When Router C receives a router poisoning from Router E, it sends an update, called a **poison reverse**, back to Router E.

This makes sure all routes on the segment have received the poisoned route information.

Assume that the network in Figure (previous problem) uses distancevector routing with the forwarding table as shown for each node. If each node periodically announces their vectors to the neighbor using the split-horizon strategy, what is the distance vector advertised in the appropriate period:

- a. from A to B? b. from C to D? c. from D to B? d. from C to A

Assume that the network in Figure 20.34 uses distance-vector routing with the forwarding table as shown for each node. If each node periodically announces their vectors to the neighbor using the poison-reverse strategy, what is the distance vector advertised in the appropriate period: a. from A to B? b. from C to D? c. from D to B? d. from C to A?

Link-State Routing

- *link-state* to define the characteristic of a link (an edge)
- The cost associated with an edge defines the state of the link

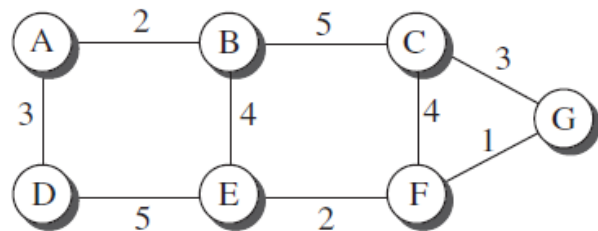
Steps:

- Discover its neighbors and build its neighbor table - Each Link State enabled router periodically sends a HELLO message on each of its links.
- Measure the cost (delay, bandwidth, etc) to each of its neighbors
- Construct and send a routing update telling all it has learned to all routers in the network
- Apply the Dijkstra algorithm to construct the shortest path to all possible destinations

Link-State Database (LSDB)

- Each node needs to have a complete *map of the network*, which means it needs to know the state of each link.
- The collection of states for all links is called the ***link-state database (LSDB)***.
- There is only one LSDB for the whole internet

Figure 20.8 Example of a link-state database



a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

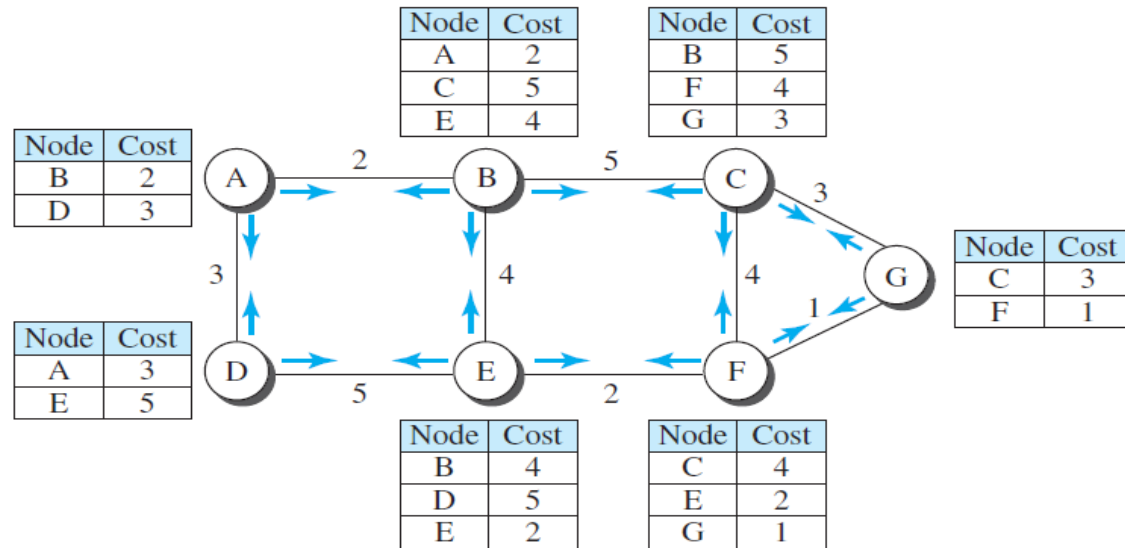
b. Link state database

How each node can create this LSDB ?

- Two phases
 - **Reliable flooding** :Tell all routers what you know about your local topology.
 - Flooding: The technique in which every incoming packet is sent on every outgoing line except from which it arrived.
 - Flooding stops somewhere in the network. (with the help of sequence number/Hop counter)
 - **Path calculation (Dijkstra's algorithm)**: Each router computes best path over complete network

To Build LSDB

Figure 20.9 *LSPs created and sent out by each node to build LSDB*

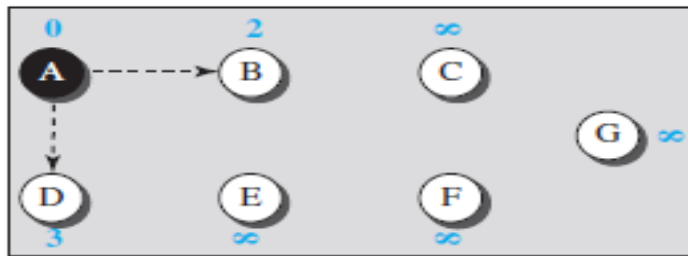


Formation of Least-Cost Trees

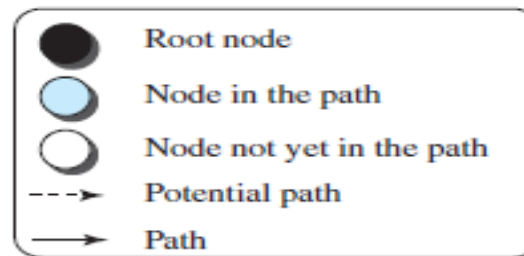
- Path calculation (Dijkstra's algorithm): Each router computes best path over complete network .
- Each node needs to run the famous **Dijkstra Algorithm**.
- **This iterative algorithm uses the following steps:**
 - **Node chooses itself as the root**, creating a tree with a single node, and sets total cost of each node based on the information in the LSDB.
 - **Node selects one node, among all nodes not in the tree, which is closest to the root**, and adds this to the tree. After this node is added to the tree, the cost of all other nodes not in the tree needs to be updated because the paths may have been changed.
 - **The node repeats step 2 until all nodes are added to the tree.**

Example

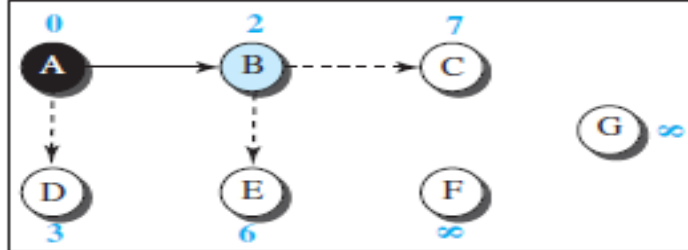
Initialization



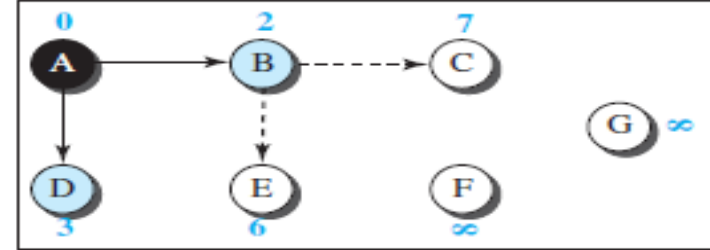
Legend



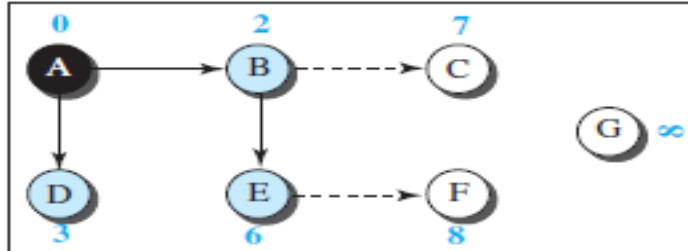
Iteration 1



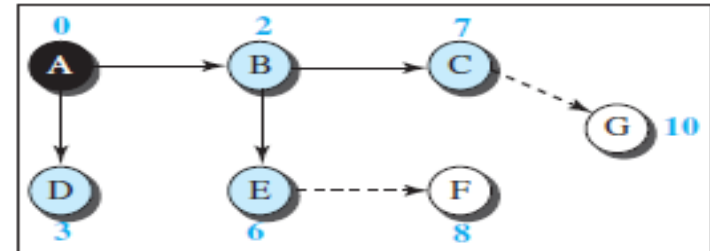
Iteration 2



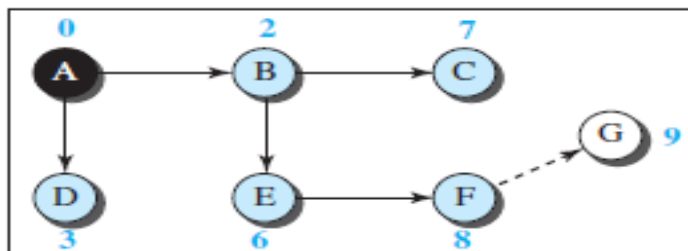
Iteration 3



Iteration 4



Iteration 5



Iteration 6

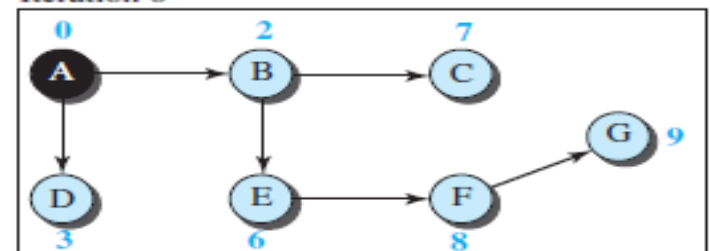
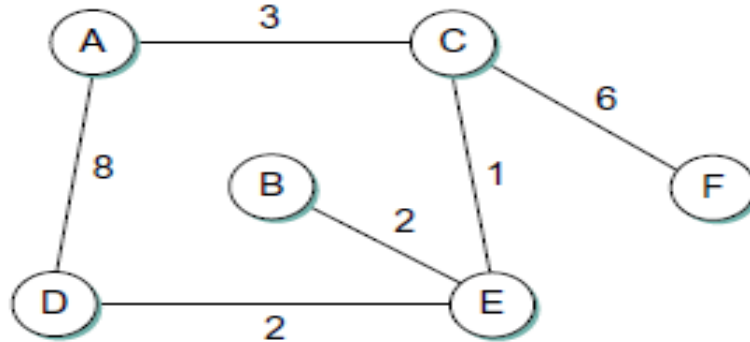


Table 20.2 *Dijkstra's Algorithm*

1	Dijkstra's Algorithm ()	
2	{	
3	// Initialization	
4	Tree = {root}	// Tree is made only of the root
5	for (y = 1 to N)	// N is the number of nodes
6	{	
7	if (y is the root)	
8	D[y] = 0	// D[y] is shortest distance from root to node y
9	else if (y is a neighbor)	
10	D[y] = c[root][y]	// c[x][y] is cost between nodes x and y in LSDB
11	else	
12	D[y] = ∞	
13	}	
14	// Calculation	
15	repeat	
16	{	
17	find a node w, with D[w] minimum among all nodes not in the Tree	
18	Tree = Tree $\cup$ {w}	// Add w to tree
19	// Update distances for all neighbors of w	
20	for (every node x, which is a neighbor of w and not in the Tree)	
21	{	
22	D[x] = min { D[x], (D[w] + c[w][x]) }	
23	}	
24	} until (all nodes included in the Tree)	
25	} // End of Dijkstra	

For the network given in Figure 3.53, show how the *link-state* algorithm builds the routing table for node D.

C	3
D	8



A	8
E	2

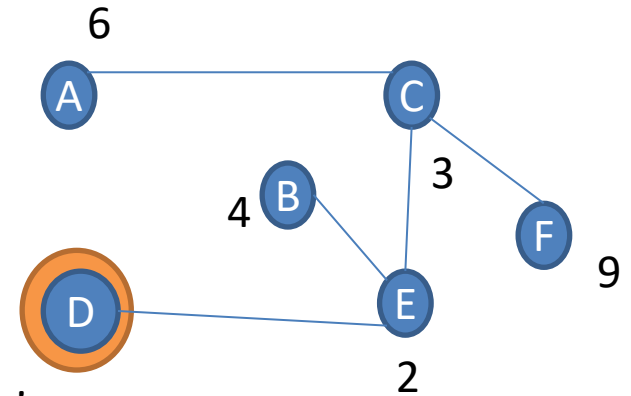
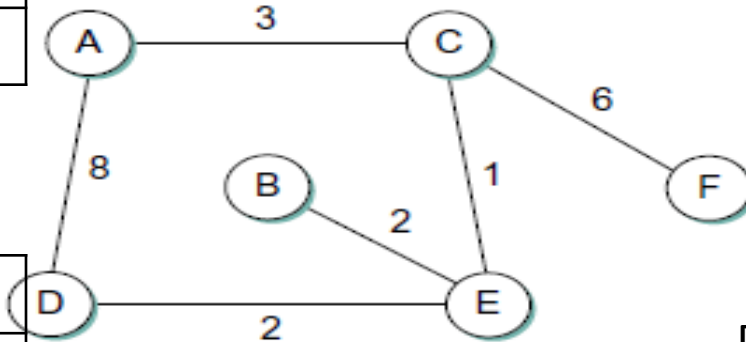
Design the LSDB

	A	B	C	D	E	F
A	0	∞	3	8	∞	∞
B	∞	0	∞	∞	2	∞
C	3	∞	∞	∞	1	6
D	8	∞	∞	∞	2	∞
E	∞	2	1	2	∞	∞
F	∞	∞	6	∞	∞	∞

For the network given in Figure 3.53, show how the *link-state* algorithm builds the routing table for node D.

C	3
D	8

A	8
E	2

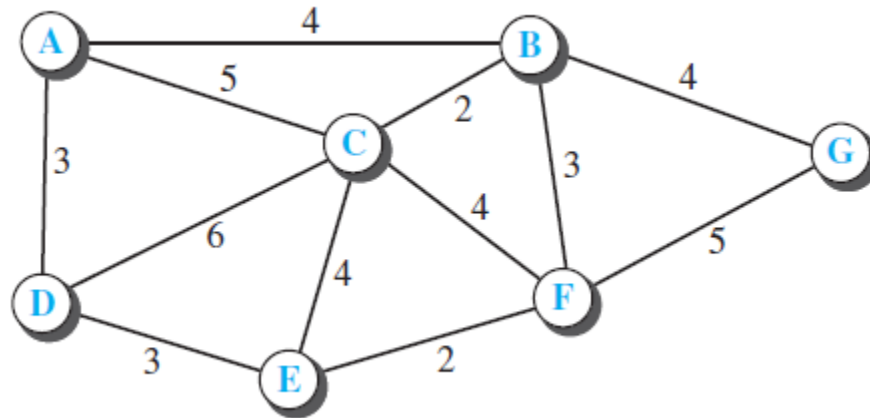


Dijkstra's

	A	B	C	D	E	F
{D}	8	∞	∞	0	2	∞
{D,E}	8	4	3	0	-	∞
{D,E,C}	6	4	-	0	-	9
{D,E,C,B}	6	-	-	0	-	9
{D,E,C,B,A}	-	-	-	-	-	9
{D,E,C,B,A,F}	-	-	-	-	-	-

	D
A	6
B	4
C	3
D	0
E	2
F	9

- Use Dijkstra's algorithm, for given Figure 20.35.



- Design the LSDB.
- Design LSP for Each router.
- Find the shortest path tree and the forwarding table for node A

Comparison

- In the distance-vector routing algorithm, each router tells its **neighbors** what it knows about the whole internet;
 - Vector will be sent to its neighboring nodes
 - Bellmen ford
-
- In the link-state routing algorithm, each router tells the whole **internet** what it knows about its neighbors.
 - Flooding is used
 - Dijkstra's with Bellmen ford

Comparison of LS and DV Routing

Message complexity

- LS: with n nodes, E links, $O(nE)$ messages sent
- DV: exchange between neighbors only

Speed of Convergence

- LS: relatively fast
- DV: convergence time varies
 - May be routing loops
 - Count-to-infinity problem

Robustness: what happens if router malfunctions?

LS:

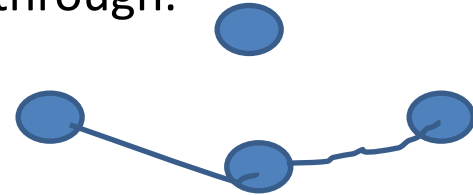
- Node can advertise incorrect *link* cost
- Each node computes only its *own* table

DV:

- DV node can advertise incorrect *path* cost
- Each node's table used by others (error propagates)

Path-Vector Routing

- Distance-vector and link-state routing are based on the least-cost goal. It always prefer least cost path. There is no option to prioritize the path.
- For example, assume that there are some routers in the internet that a sender wants to prevent its packets from going through.



- In other words, the least-cost goal, applied by LS or DV routing, does not allow a sender to apply specific policies to the route a packet may take.

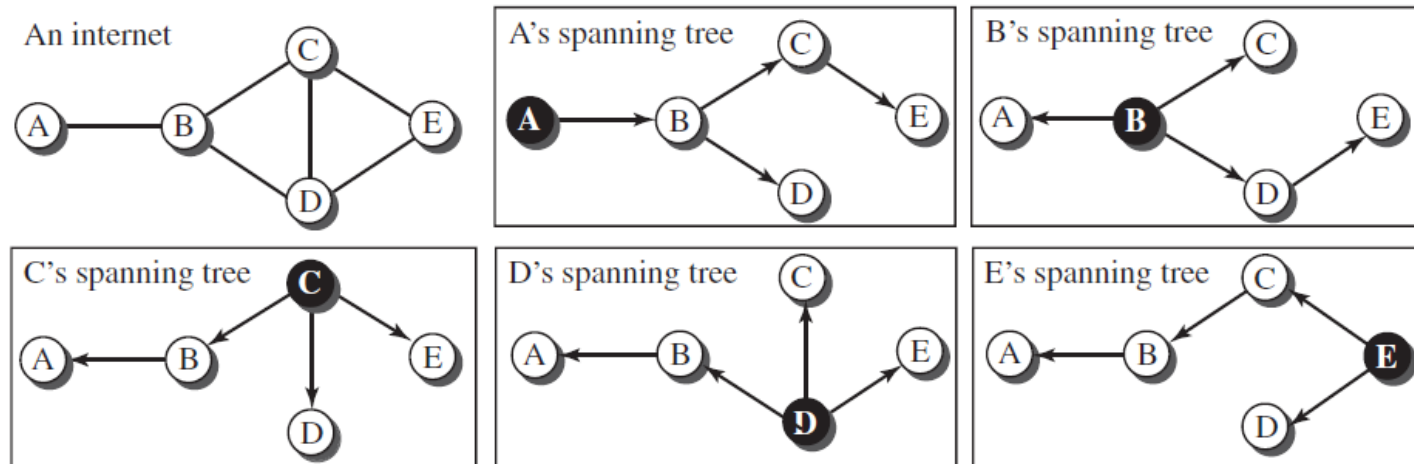
Path-Vector (PV) routing

- The best route is determined by the source **using the policy** it imposes on the route.
- The source can control the path
- Designed to route a packet between ISPs
- Policy could be
 - Secure path
 - Less delay
 - to avoid some nodes as the middle node in a route
 - minimum number of nodes to be visited

Spanning Trees

- In path-vector routing, the path from a source to all destinations is also determined by the **best spanning tree**.
- best spanning tree - is not the least-cost tree; it is the tree determined by the source when it imposes its own policy

Figure 20.11 *Spanning trees in path-vector routing*

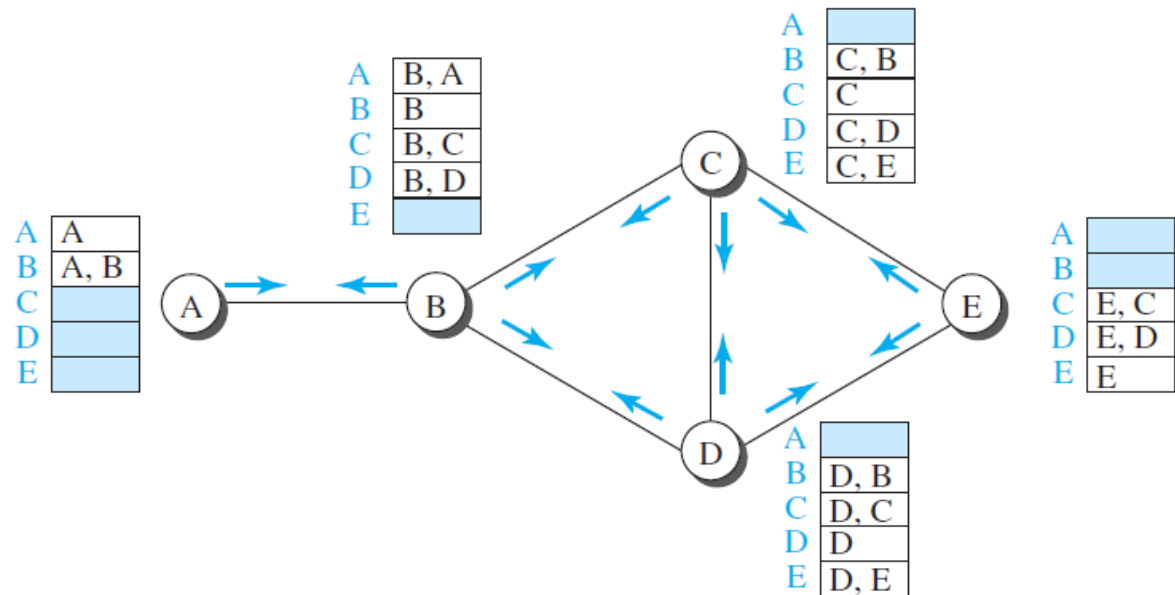


- Each source has created its own spanning tree that meets its policy.
- Policy:
- Use the minimum number of nodes to reach a destination
 - The spanning tree selected by A and E is such that the communication does not pass through D as a middle node

Creation of Spanning Trees

- Path-vector routing, like distance-vector routing, is an asynchronous and distributed routing algorithm
- When a node is booted, it creates a path vector based on the information it can obtain about its **immediate neighbor** -greeting messages to its immediate neighbors

Figure 20.12 *Path vectors made at booting time*



we do not mean that all of these tables are created simultaneously. They are created when each node is booted

Creation of Spanning Trees

- Each node, after the creation of the initial path vector, sends it to all its **immediate neighbors**.
- Each node, when it receives a path vector from a neighbor, updates its path vector using an equation similar to the Bellman-Ford, but applying its own policy instead of looking for the least cost.

$$\text{Path}(x, y) = \text{best} \{ \text{Path}(x, y), [(x + \text{Path}(\mathbf{v}, y))] \} \quad \text{for all } \mathbf{v}'\text{'s in the internet.}$$

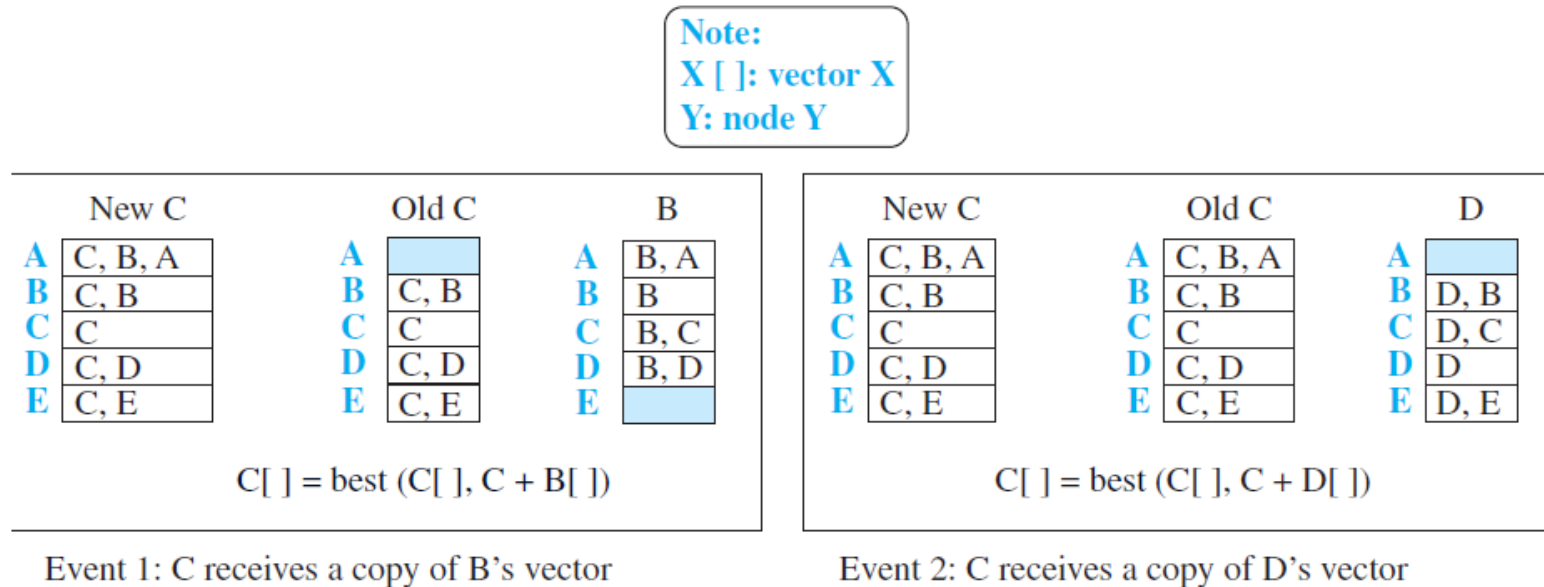
Policy

By selecting the best of multiple paths

x does not want to visit itself when it selects a path to y - to avoid a loop in the path

Creation of Spanning Trees

Figure 20.13 *Updating path vectors*



The path vector of node C after two events

- In the first event, node C receives a copy of B's vector, which improves its vector
- In the second event, node C receives a copy of D's vector, which does not
- change its vector.

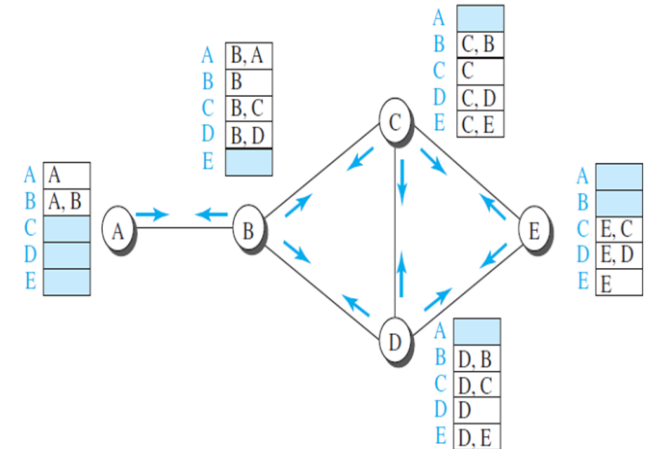
Table 20.3 Path-vector algorithm for a node

```

1 Path_Vector_Routing ( )
2 {
3     // Initialization
4     for (y = 1 to N)
5     {
6         if (y is myself)
7             Path[y] = myself
8         else if (y is a neighbor)
9             Path[y] = myself + neighbor node
10        else
11            Path[y] = empty
12    }
13    Send vector {Path[1], Path[2], ..., Path[y]} to all neighbors
14    // Update
15    repeat (forever)
16    {
17        wait (for a vector Pathw from a neighbor w)
18        for (y = 1 to N)
19        {
20            if (Pathw includes myself)
21                discard the path                // Avoid any loop
22            else
23                Path[y] = best {Path[y], (myself + Pathw[y])}
24        }
25        If (there is a change in the vector)
26            Send vector {Path[1], Path[2], ..., Path[y]} to all neighbors
27    }
28 } // End of Path Vector

```

Figure 20.12 Path vectors made at booting time



Next Generation IP

IP addresses

Internet Protocol Version 4 (IPv4)

- 32-bit numeric addressing scheme
- 2^{32} IP addresses -around 4.29 billion IP addresses
- Address depletion problem – IoT, smart phones, more internet connected devices

Internet Protocol Version 5(IPv5)

- IPv5 never became an official protocol.
- The ST/IPv5 internet protocol was developed as a means of [streaming video](#) and voice data by Apple, NeXT, and Sun Microsystems, and it was experimental.
- serve as a foundation for the development of technologies like voice-over-IP
- IPv5 used IPv4's 32-bit addressing, which eventually became a problem

Internet Protocol Version 6(IPv6)

- An IPv6 address is 128 bits or 16 bytes(octets) long
- 2^{128} —340,282,366,920,938,463,463,374,607,431,768,211,456 IP addresses
- An example of an IPv6 address is 2001:0db8:0000:0000:1234:0ace:6006:001e.

IPv6 Representation

Binary (128 bits)	1111111011110110 ... 1111111100000000
Colon Hexadecimal	FEF6:BA98:7654:3210:ADEF:BBFF:2922:FF00

- **hexadecimal notation** (or *colon hex* for short) divides the address into eight sections, each made of four hexadecimal digits separated by colons.

Abbreviation

- Even in hexadecimal format, it is very long, many of the digits are zeros
- The leading zeros of a section can be omitted.
 - 0074 can be written as 74, 000F as F, and 0000 as 0
 - 3210 cannot be abbreviated
- abbreviation, often called **zero compression**
- We can remove all the zeros and replace them with a double semicolon.

FDEC:0:0:0:0:BBFF:0:FFFF	→	FDEC::BBFF:0:FFFF
--------------------------	---	-------------------

Representation

Mixed Notation

- Mixed representation of an IPv6 address: colon hex and dotted decimal notation.
- This is appropriate during the **transition period in which an IPv4 address is embedded in an IPv6 address** (as the rightmost 32 bits).
- This happens when all or most of the leftmost sections of the IPv6 address are 0s.
 - Eg: (::130.24.24.18)
 - zero compression shows that all $128-32=96$ leftmost bits of the address are zeros.

CIDR Notation

- IPv6 uses hierarchical addressing (network part, host part, subnet...)
- In IPv6 space instead of class here we **have prefixes**.

FDEC::BBFF:0:FFFF/60

Question

- Show the unabbreviated colon hex notation for the following IPv6 addresses:
 - a. An address with 64 0s followed by 64 1s.
 - b. An address with 128 0s.
 - c. An address with 128 1s.
 - d. An address with 128 alternative 1s and 0s.
- **Solution**
 - a. 0000:0000:0000:0000:FFFF:FFFF:FFFF:FFFF (each octet is $4*4 = 16$ bits)
 - b. 0000:0000:0000:0000:0000:0000:0000:0000
 - c. FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF
 - d. AAAA:AAAA:AAAA:AAAA:AAAA:AAAA:AAAA:AAAA

Question

- Show abbreviations for the following addresses:
 - a. 0000:0000:FFFF:0000:0000:0000:0000:0000
 - b. 1234:2346:0000:0000:0000:0000:0000:1111
 - c. 0000:0001:0000:0000:0000:0000:1200:1000
 - d. 0000:0000:0000:0000:0000:FFFF:24.123.12.6
- Solution
 - a. 0:0:FFFF::
 - b. 1234:2346::1111
 - c. 0:1::1200:1000
 - d. ::FFFF:24.123.12.6

Question

- Decompress the following addresses and show the complete unabbreviated IPv6 address:
 - a. **1111::2222**
 - b. **::**
 - c. **0:1::**
 - d. **AAAA:A:AA::1234**

- Solution**

- a. **1111:0000:0000:0000:0000:0000:0000:2222**
 - b. **0000:0000:0000:0000:0000:0000:0000:0000**
 - c. **0000:00001:0000:0000:0000:0000:0000:0000**
 - d. **AAAA:000A:00AA:0000:0000:0000:0000:1234**

Address Space

- IPv6 contains 2^{128} addresses
- The size of the space: 340, 282, 366, 920, 938, 463, 374, 607, 431, 768, 211, 456.
- The number of people on the earth is soon to be 234 (more than 16 billion).
- Each person can have 2^{88} addresses to use.
- Address depletion in this version is impossible.

Three Address Types

- In IPv6, a destination address can belong to one of three categories:
 - Unicast
 - Anycast
 - Multicast.

Three Address Types

Unicast Address:

- A unicast address defines a single interface (computer or router).
- The packet sent to a unicast address will be routed to the intended recipient.

Anycast Address:

- An **anycast address** defines a group of computers that all share a single address
- A packet with an anycast address is delivered to only one member of the group, the most reachable one
- for example, when there are several servers that can respond to an inquiry. The request is sent to the one that is most reachable

Multicast Address:

- A multicast address also defines a group of computers
- In anycasting, only one copy of the packet is sent to one of the members of the group;
- In multicasting each member of the group receives a copy.

Address Space Allocation

- The address space of IPv6 is divided into several blocks of varying size and each block is allocated for a special purpose. Most of the blocks are still unassigned and have been set aside for future use. Table 22.1 shows only the assigned blocks.

Table 22.1 *Prefixes for assigned IPv6 addresses*

<i>Block prefix</i>	<i>CIDR</i>	<i>Block assignment</i>	<i>Fraction</i>
0000 0000	0000::/8	Special addresses	1/256
001	2000::/3	Global unicast	1/8
1111 110	FC00::/7	Unique local unicast	1/128
1111 1110 10	FE80::/10	Link local addresses	1/1024
1111 1111	FF00::/8	Multicast addresses	1/256

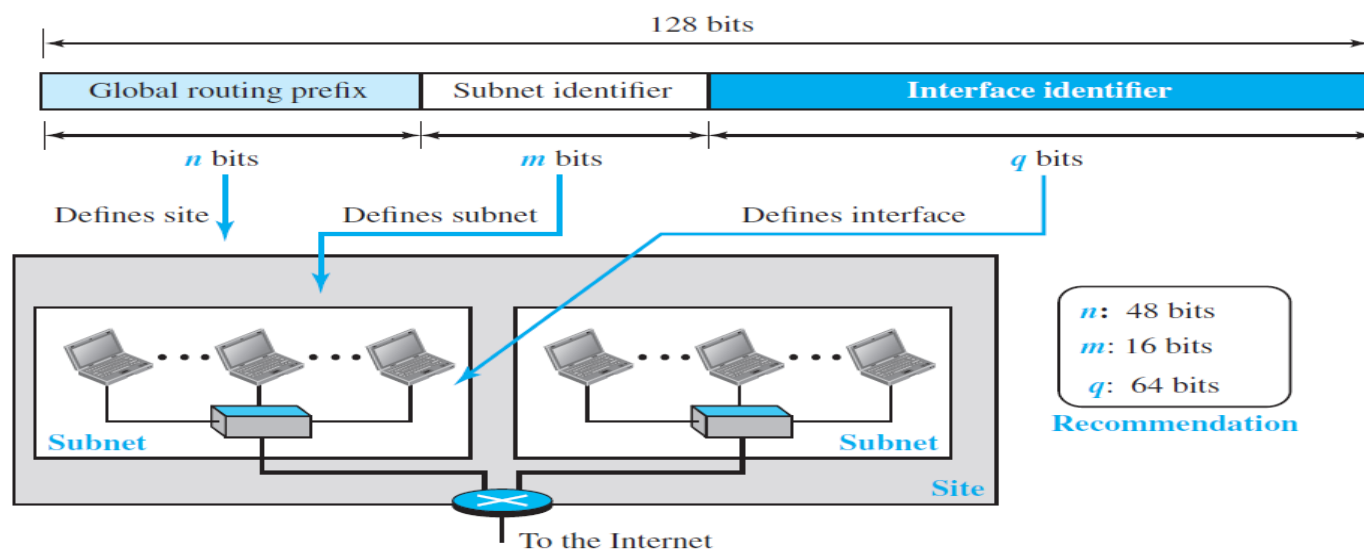
Global Unicast Addresses

- The block in the address space that is used for unicast (one-to-one) communication between two hosts in the Internet

001	2000::/3	Global unicast	1/8
-----	----------	----------------	-----

- Three leftmost bits are the same for all addresses in this block (001).
- 0010000000000000::
- The size of this block is 2^{125} bits
- An address in this block is divided into three parts:
- global routing prefix* (n bits), *subnet identifier* (m bits), and *interface identifier* (q bits)

Figure 22.1 Global unicast address



Global Unicast Addresses

n: 48 bits

m: 16 bits

q: 64 bits

Recommendation

- ***n: 48 bits*** Since the first three bits in this part are fixed (001), the rest of the 45 bits can be defined for up to 2^{45} sites (a private organization or an ISP).
 - The global routers in the Internet route a packet to its destination site based on the value of *n*
- ***m: 16 bits*** define a subnet in an organization. An organization can have up to $2^{16} = 65,536$ subnets
- ***Q: 64 bits*** define the interface identifier. The interface identifier is similar to hostid in IPv4 addressing

2001:1234:ABCD:5678:0221:2FFF:FEB5:6E10

Mapping process (IPv6 to MAC)

IPv4 : 32 bits and MAC:48 bits

- In IPv4 addressing, there is not a specific relation between the hostid (at the IP level) and link-layer address (at the data-link layer)

IPv6 : 128 bits and MAC:48 bits

The IPv6 addressing allows this relationship.

- A link-layer address whose length is less than 64 bits can be embedded as the whole or part of the interface identifier

Two common linklayer addressing schemes:

- The 48-bit link-layer address defined by Ethernet.
- The 64-bit extended unique identifier (EUI-64) defined by IEEE

Mapping EUI-64

Figure 22.2 Mapping for EUI-64

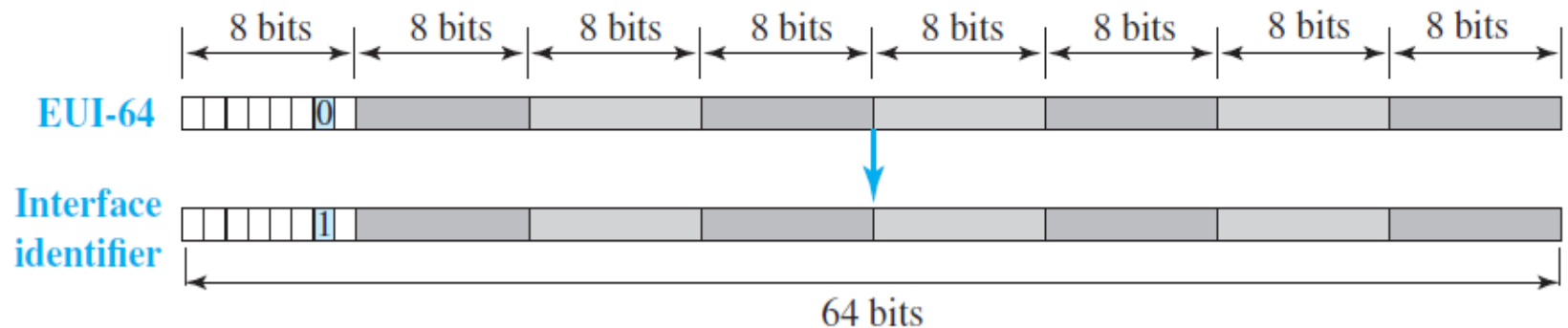
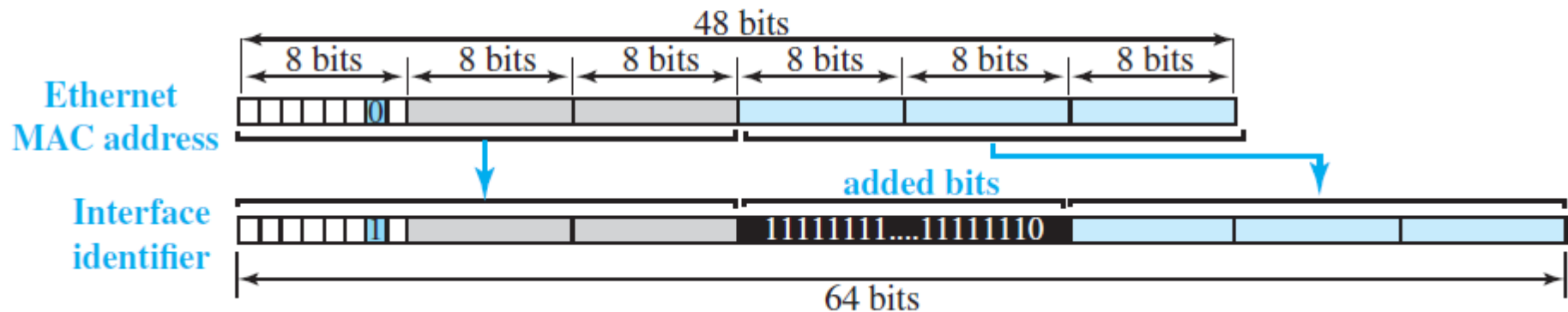


Figure 22.3 Mapping for Ethernet MAC



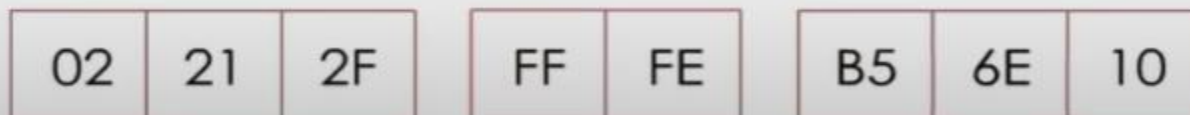
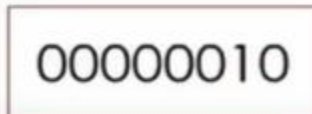
Mapping EUI-64



48-Bit Mac Address



64-Bit EUI



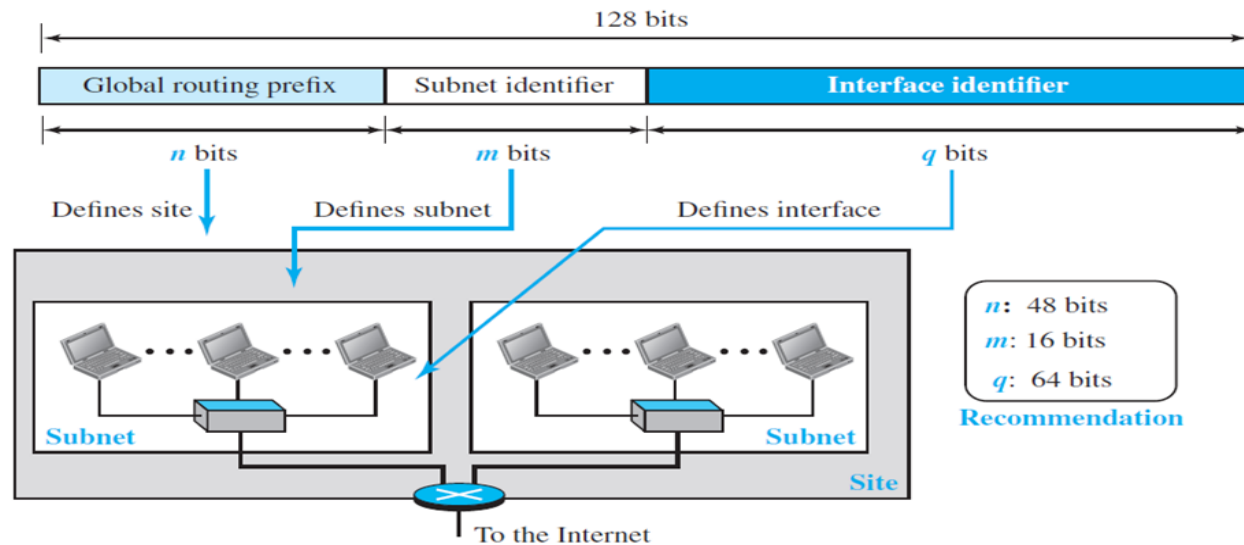
EUI-64 Part

Note

the first 64 bit part,
which includes global routing
prefix and subnet ID,
comes from your default gateway,

the second 64-bit is generated
by your device itself.

Figure 22.1 *Global unicast address*



Microsoft Windows uses a
random value to generate
the second portion.

Cisco IOS, Linux,
and unix systems
use EUI-64 method.

Example 22.3

Using the format we defined for Ethernet addresses, find the interface identifier if the Ethernet physical address is **(F5-A9-23-14-7A-D2)₁₆**.

Solution

We only need to change the seventh bit of the first octet from 0 to 1, insert two octets **FFFE₁₆** and change the format to colon hex notation. The result is **F7A9:23FF:FE14:7AD2** in colon hex.

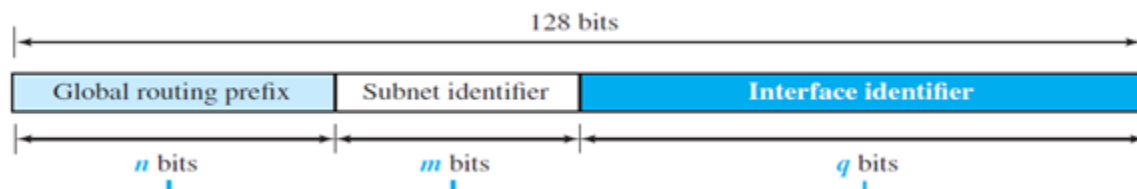
Example 22.4

An organization is assigned the block 2000:1456:2474/48. What is the IPv6 address of an interface in the third subnet if the IEEE physical address of the computer is (F5-A9-23-14-7A-D2)₁₆?

Solution

The interface identifier for this interface is **F7A9:23FF:FE14:7AD2** (see Example 22.3). If we append this identifier to the global prefix and the subnet identifier, we get:

2000:1456:2474:0003:F7A9:23FF:FE14:7AD2/128



n : 48 bits

m : 16 bits

q : 64 bits

Recommendation

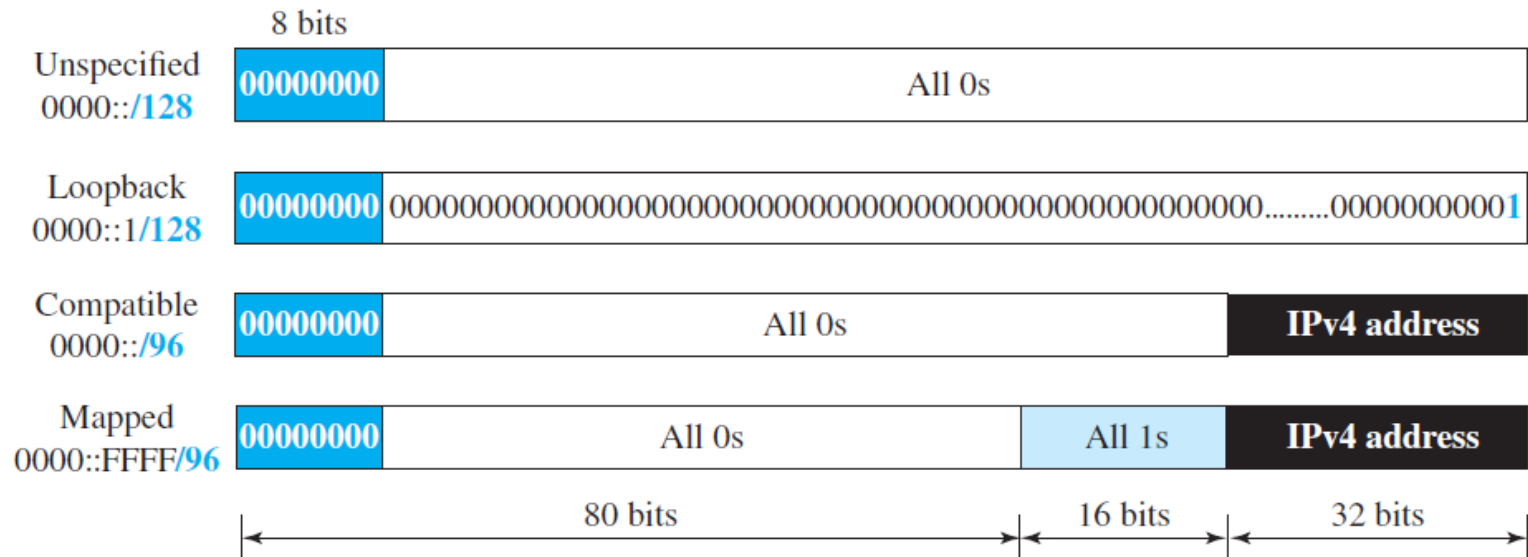
Problem

Assume a host with Ethernet address in hexadecimal (F5-A9-27-90-B7-40) has joined the network. What would be its global unicast address if the global unicast prefix of the organization is 6C45:2434:7845 and the subnet identifier is E467

Special Addresses

- Special Addresses- that use the prefix (**0000::/8**) are reserved

Figure 22.4 *Special addresses*



Special Addresses

- **Unspecified address** is a subblock containing only one address
 - used during **bootstrap** when a host does not know its own address and wants to send an inquiry to find it (DHCP)
- **loopback address** also consists of one address

During the transition from IPv4 to IPv6, hosts can use their IPv4 addresses embedded in IPv6 addresses:

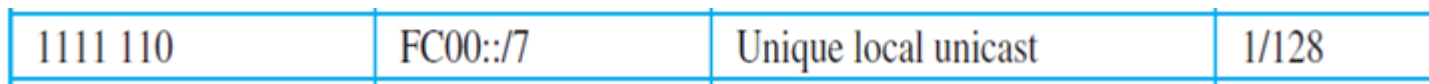
- **compatible address** is an address of 96 bits of zero followed by 32 bits of IPv4 address
 - used when a computer using IPv4 wants to send a message to another computer using IPv6
- **mapped address** is used when a computer already migrated to version 6 wants to send an address to a computer still using version 4.

Other Assigned Blocks

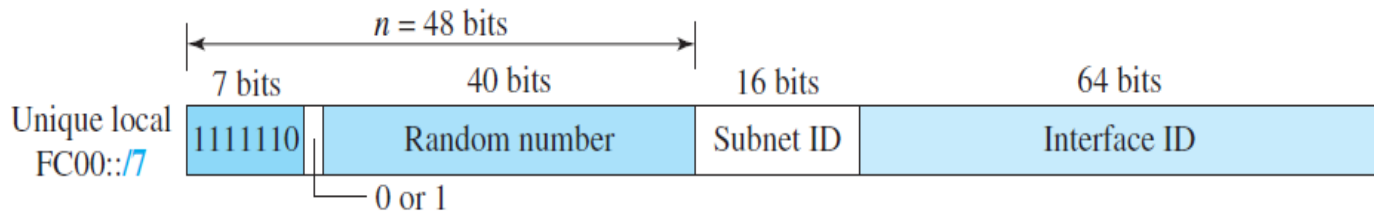
unique local unicast block: can be privately created and used by a site

The packet carrying this type of address as the destination address is not expected to be routed

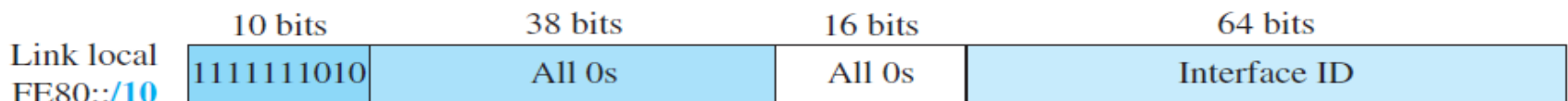
- This type of address has the identifier 1111 110, the next bit can be 0 or 1



- The next 40 bits are selected by the site using a 40-bit random generator

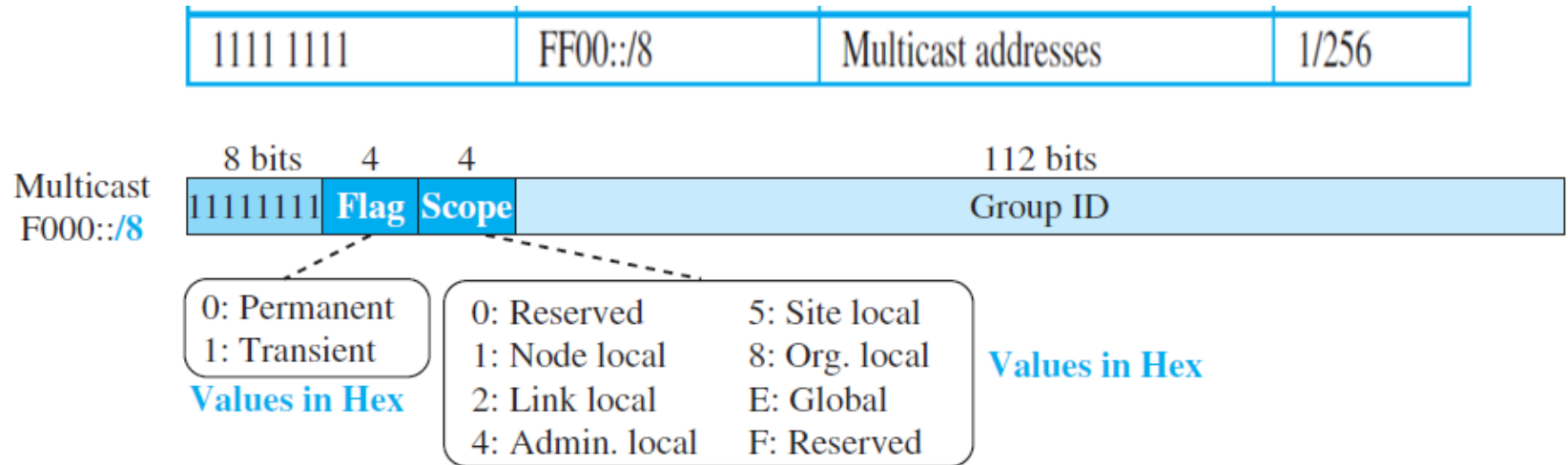


link local block: address in this block can be used as a private address in a network



Other Assigned Blocks

- Multicast addresses**



- A permanent group address is defined by the Internet authorities and can be accessed at all times.
- A transient group address, on the other hand, is used only temporarily.
 - Ex: Systems engaged in a teleconference

Autoconfiguration

In IPv6, DHCP protocol can still be used to allocate an IPv6 address to a host, but a host can also configure itself.

When a host in IPv6 joins a network, it can configure itself using the following process:

1. The host first creates a **link local address** for itself

10-bit link local prefix (1111 1110 10) + 54 zeros + 64-bit interface identifier = 128 bits

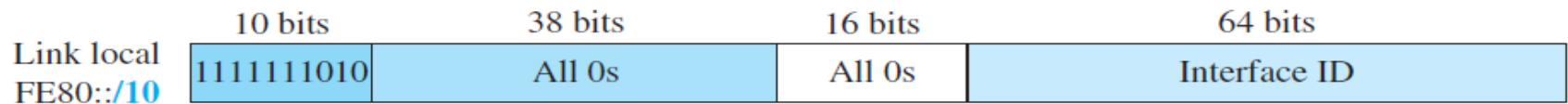
2. The host then **tests to see** if this link local address is **unique and not** used by other hosts.

- the host sends a ***neighbor solicitation message*** and waits for a *neighbor advertisement message*

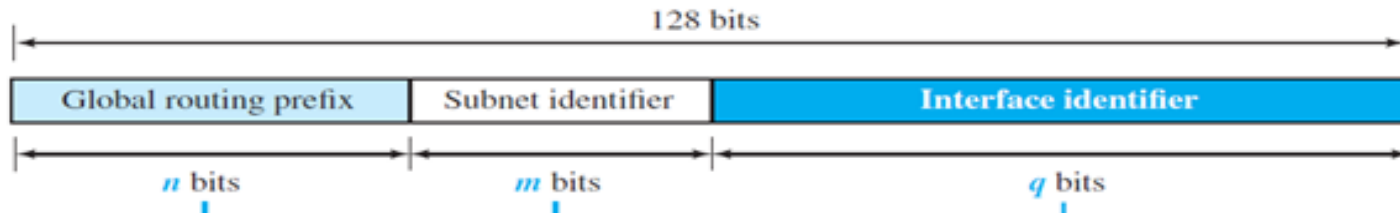
3. If the uniqueness of the **link local address is passed** the host stores this address as its link local address (for private communication), but it still needs a **global unicast address**.

- The host then sends a ***router solicitation message*** to a local router.
- If there is a router running on the network, the host receives a ***router advertisement message*** that includes the global unicast prefix and the subnet prefix that the host needs to add to its interface identifier to generate its global unicast address.

link local address - *neighbor solicitation message*



global unicast address- router solicitation message *and router advertisement message*



Autoconfiguration Example

Example 22.5

Assume a host with Ethernet address (**F5-A9-23-11-9B-E2**)₁₆ has joined the network. What would be its global unicast address if the global unicast prefix of the organization is 3A21:1216:2165 and the subnet identifier is A245:1232?

Solution

The host first creates its interface identifier as **F7A9:23FF:FE11:9BE2** using the Ethernet address read from its card. The host then creates its link local address as:

FE80::F7A9:23FF:FE11:9BE2

Assuming that this address is unique, the host sends a router solicitation message and receives the router advertisement message that announces the combination of global unicast prefix and the subnet identifier as 3A21:1216:2165:A245:1232. The host then appends its interface identifier to this prefix to find and store its global unicast address as:

3A21:1216:2165:A245:1232:F7A9:23FF:FE11:9BE2

THE IPv6 PROTOCOL

Better header format. IPv6 uses a new header format in which ***options are separated from the base header*** and inserted, when needed, between the base header and the data. This simplifies and speeds up the routing process because most of the options do not need to be checked by routers.

New options. IPv6 has new options to allow for additional functionalities.

Allowance for extension. IPv6 is designed to allow the extension of the protocol if required by new technologies or applications.

Support for resource allocation. In IPv6, the type-of-service field has been removed, but two new fields, traffic class and flow label, have been added to enable the source to request special handling of the packet. This mechanism can be used to **support traffic such as real-time audio and video.**

Support for more security. The encryption and authentication options in IPv6 provide confidentiality and integrity of the packet.

Packet Format

Figure 22.6 IPv6 datagram

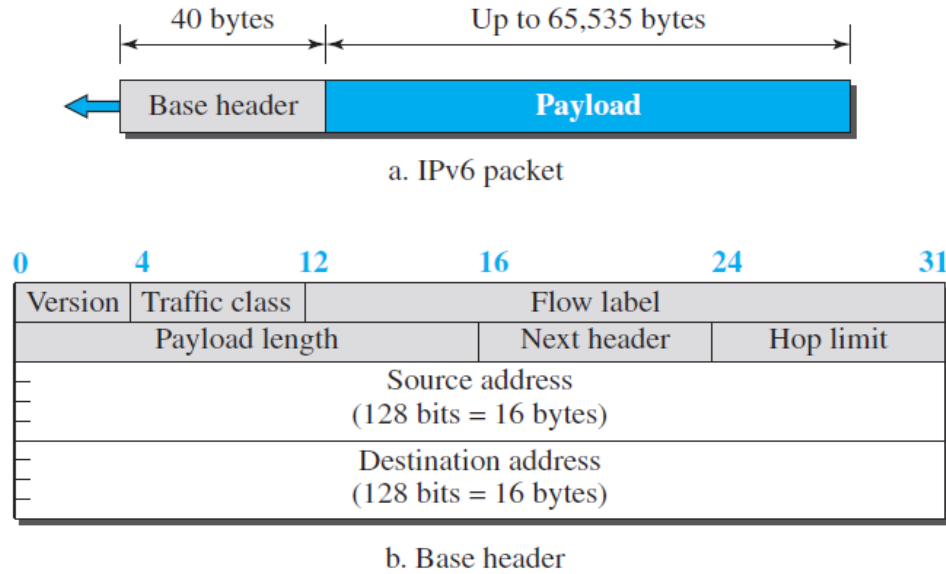
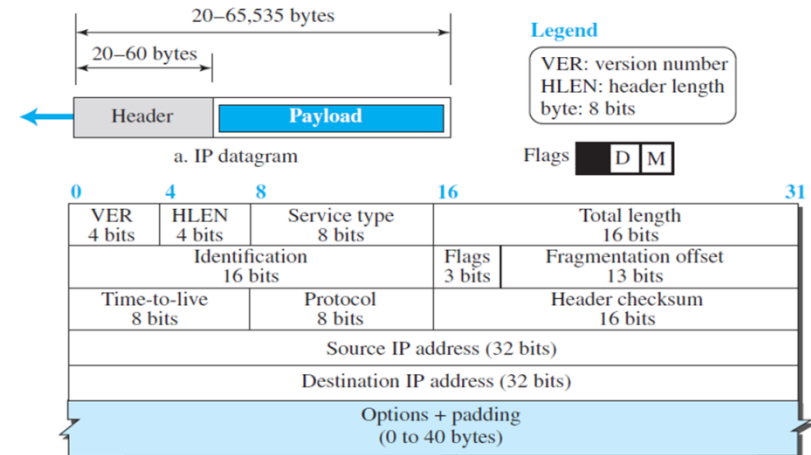
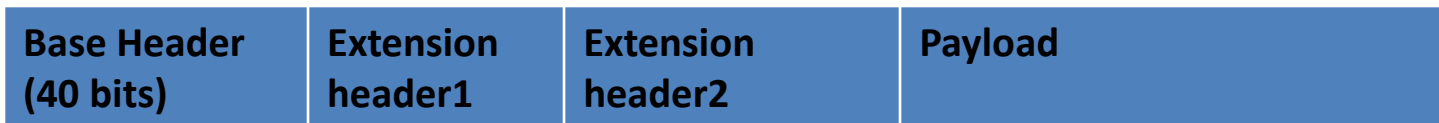


Figure 19.2 IP datagram



Packet Format

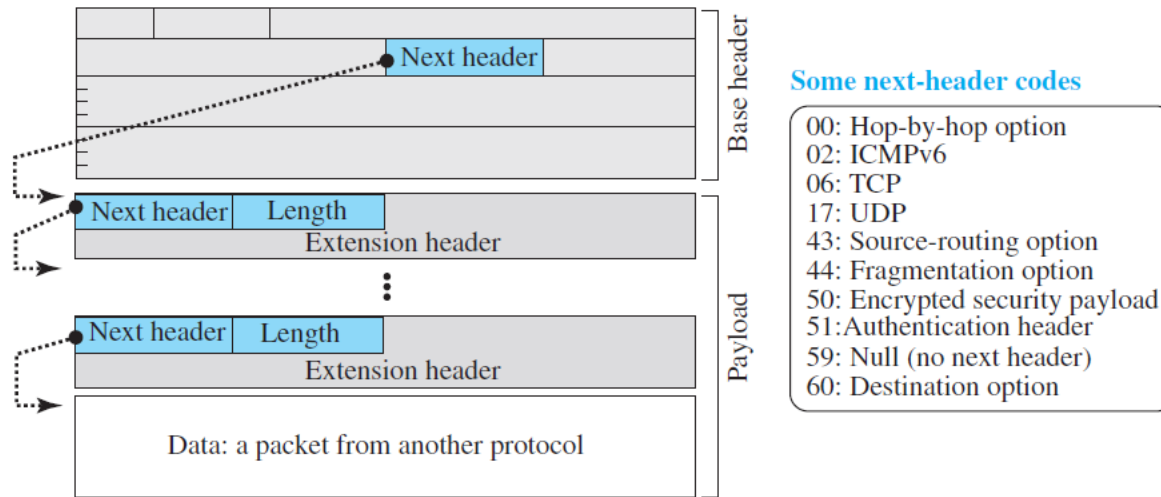
- **Version:** For IPv6, the value is 6.
- **Traffic class:** The Traffic Class field indicates the IPv6 packet's class or **priority**. used to distinguish different payloads with different delivery requirements
 - Precedence (to set priority- whether the packet is routine(normal) or immediate) . During congestion –priority can be set to forward the packet
 - Type of Service(Delay , Throughput, Reliability, Cost, Reserved)
- **Flow label:** The Flow Label field indicates that this packet belongs to a specific sequence of packets between a source and destination (virtual circuit path –same direction in all the packets)
- Designed to provide special handling for a particular flow of data. The flow label is used for prioritized delivery, such as delivery needed by real-time data (voice and video).
- **Payload length:** defines the length of the IP datagram excluding the header
- **Next header:** defining the type of the first extension header (if present) or the type of the data that follows the base header in the datagram (optional header – extra functionalities)



- **Hop limit:** serves the same purpose as the TTL field in IPv4
- **Source and destination addresses:** 128 bits IP address

Next header

Figure 22.7 *Payload in an IPv6 datagram*



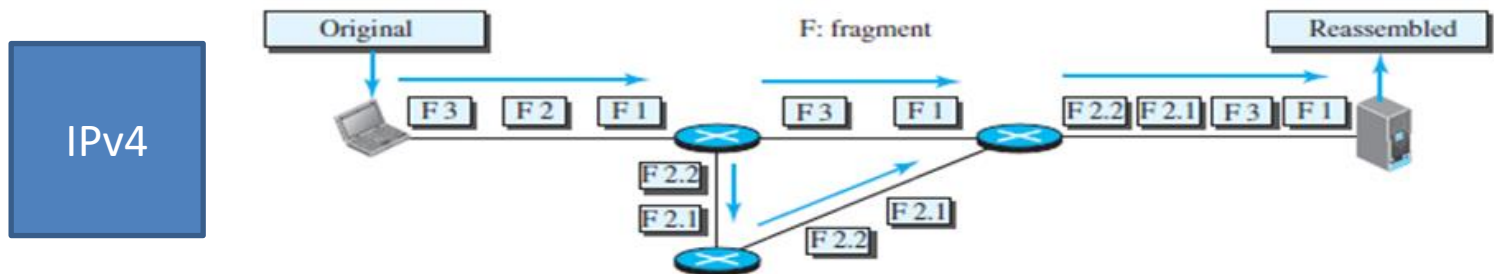
- Each extension header has two mandatory fields, next header and the length followed by information related to the particular option.
- Note that each next header field value (code) defines the type of the next header (hop-by-hop option, sourcerouting option, . . .); the last next header field defines the protocol (UDP, TCP, . . .)

Concept of Flow and Priority in IPv6

- Flow label can be used to support the transmission of real-time audio and video
- A flow label can be used to speed up the processing of a packet by a router.
- A router that supports the handling of flow labels has a flow label table
 - each entry defines the services required by the corresponding flow label.
- When a router receives a packet, instead of consulting the forwarding table and going through a routing algorithm to define the address of the next hop, it can easily look in a flow label table for the next hop.
- Real-time audio or video – requires resources such as high bandwidth, large buffers, long processing time, and so on
- A process can make a reservation for these resources beforehand to guarantee that real-time data will not be delayed due to a lack of resources.

Fragmentation and Reassembly

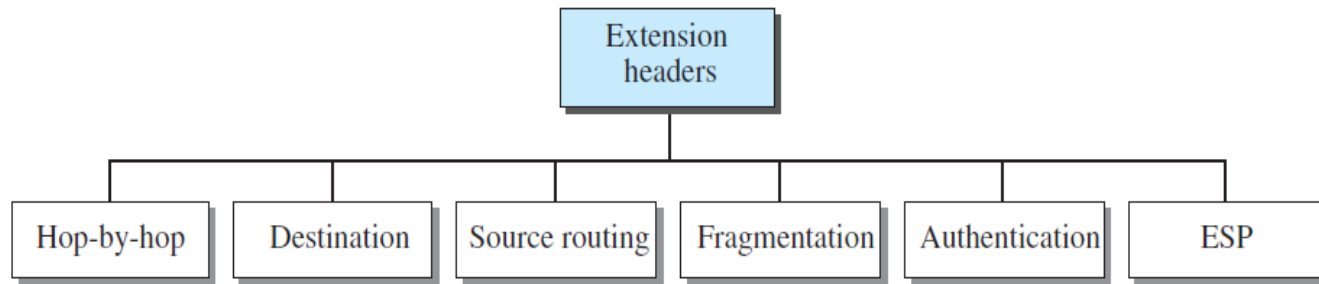
- IPv6 datagrams can be fragmented **only by the source**, not by the routers; the reassembly takes place at the destination. **Path MTU Discovery technique** to find the smallest MTU supported by any network on the path
- In IPv6, the source can check the size of the packet and make the decision to fragment the packet or not.
- The fragmentation of packets at routers is not allowed to speed up the processing of packets in the router.
 - a router needs a lot of processing
- When a router receives the packet, it can check the size of the packet and drop it if the size is larger than allowed by the MTU of the network ahead



Extension Header

- To give more functionality to the IP datagram, the base header can be followed by up to six **extension headers**

Figure 22.8 *Extension header types*



Hop-by-Hop Option

The hop-by-hop option is used when the source needs to pass information to all routers visited by the datagram

Routers must be informed about certain management, debugging, or control functions. Or, if the length of the datagram is more than the usual 65,535 bytes, routers must have this information

- ***Destination Option:***The **destination option** is used when the source needs to pass information to the destination only. Intermediate routers are not permitted access to this information
- **Source Routing:** The source routing extension header combines the concepts of the strict source route and the loose source route options of IPv4.
- **Fragmentation:**In IPv6, only the original source can fragment
- A source must use a **Path MTU Discovery technique** to find the smallest MTU supported by any network on the path. The source then fragments using this knowledge

- **Authentication**

The **authentication** extension header has a dual purpose:

validates the message sender :the receiver can be sure that a message is from the genuine sender and not from an imposter.

ensures the integrity of data :The latter is needed to check that the data is not altered in transition by some hacker

- **Encrypted Security Payload**

The encrypted security payload (ESP) is an extension that provides confidentiality and guards against eavesdropping.

Comparison of Options between IPv4 and IPv6

- Options In IPv4 is replaced as extension header
- The fragmentation fields in the base header section of IPv4 have moved to the fragmentation extension header in IPv6.
- The authentication extension header is new in IPv6.
- The encrypted security payload extension header is new in IPv6.
- The timestamp option is not implemented because it was not used.
- The source route option is called the *source route extension header* in IPv6
- The no-operation and end-of-option options in IPv4 are replaced by Pad1 and PadN options in IPv6

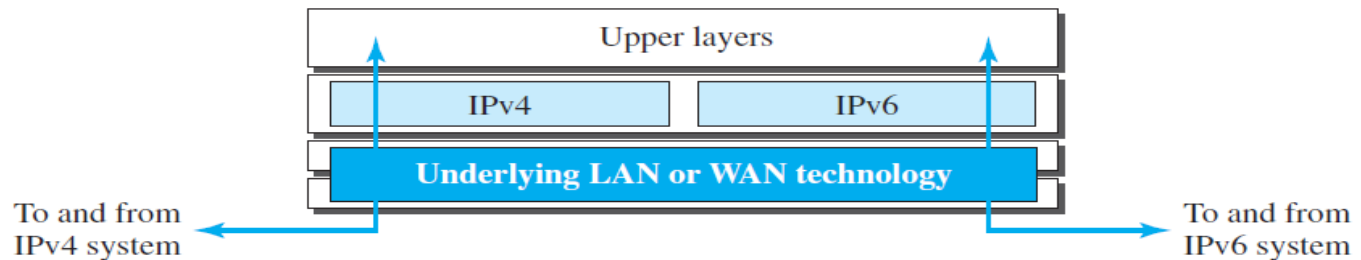
TRANSITION FROM IPv4 TO IPv6

- Three strategies have been devised for transition: dual stack, tunneling, and header translation

Dual stack

- A station must run IPv4 and IPv6 simultaneously until all the Internet uses IPv6.

Figure 22.11 *Dual stack*

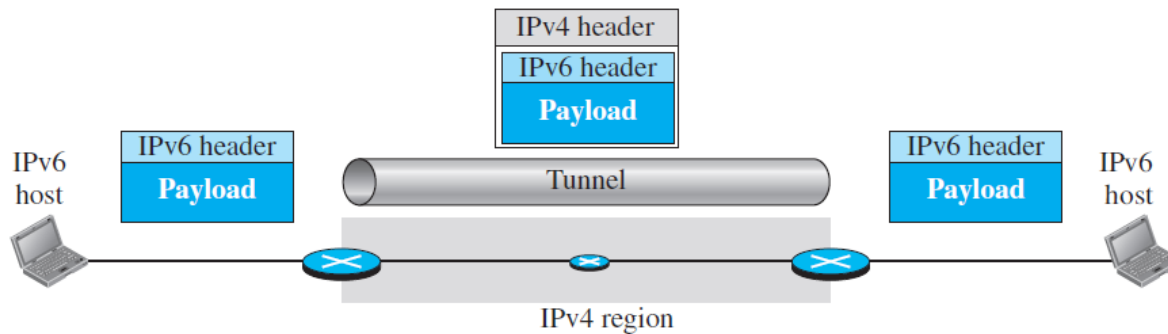


- To determine which version to use when sending a packet to a destination, the source host queries the DNS.
- If the DNS returns an IPv4 address, the source host sends an IPv4 packet. If the DNS returns an IPv6 address, the source host sends an IPv6 packet.

Tunneling

- **Tunneling** is a strategy used when two computers using IPv6 want to communicate with each other and the packet must pass through a region that uses IPv4
- The IPv6 packet is encapsulated in an IPv4 packet when it enters the region, and it leaves its capsule when it exits the region.
- The IPv4 packet is carrying an IPv6 packet as data, the protocol value is set to 41

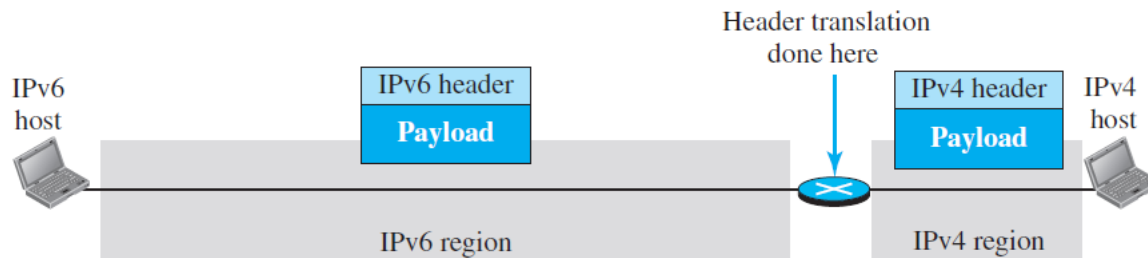
Figure 22.12 *Tunneling strategy*



Header Translation

- **Header translation** is necessary when the majority of the Internet has moved to IPv6 but some systems still use IPv4.
- The sender wants to use IPv6, but the receiver does not understand IPv6.
- In this case, the header format must be totally changed through header translation

Figure 22.13 *Header translation strategy*



Use of IP Addresses

During the transition a host may need to use two addresses, IPv4 and IPv6. When the transition is complete, IPv4 addresses should disappear.